

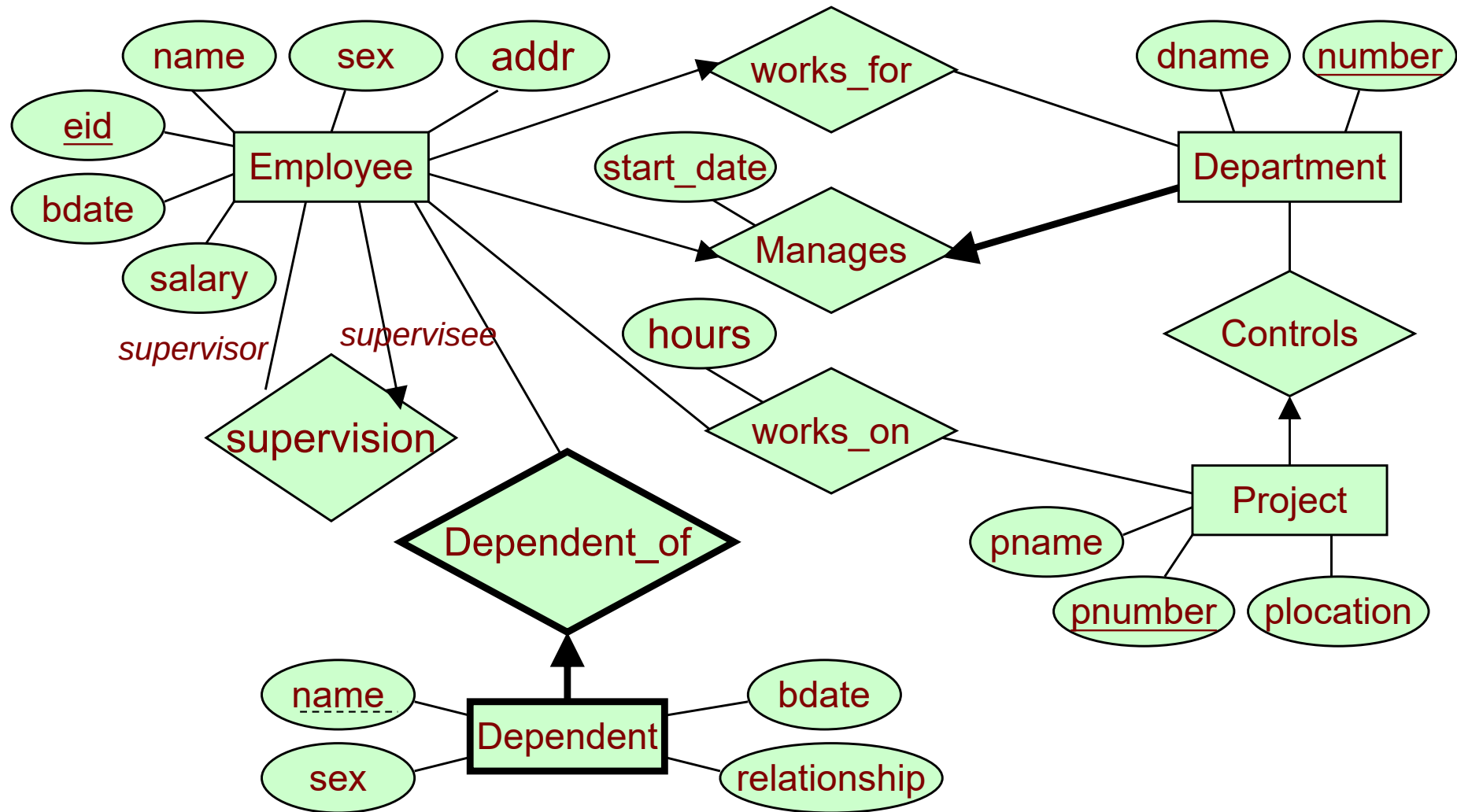
Outline

- Relational Model
 - Relational Model Concepts
 - Relational Database Schemas
- **ER-to-Relational Mapping**
 - **Translating traditional ER diagrams**
 - Translating Class Hierarchy

ER-to-Relational Mapping

- Typically, database designers begin with the ER model, which is very **expressive** and **user-friendly** to human
- Then, the ER model is mapped to the relational model for DBMS manipulations
- Database queries and updates will be written according to the relational model

Running Example



ER to Relational Model

1. Convert entities first

- From Strong Entity to Weak Entity

2. Convert relationships

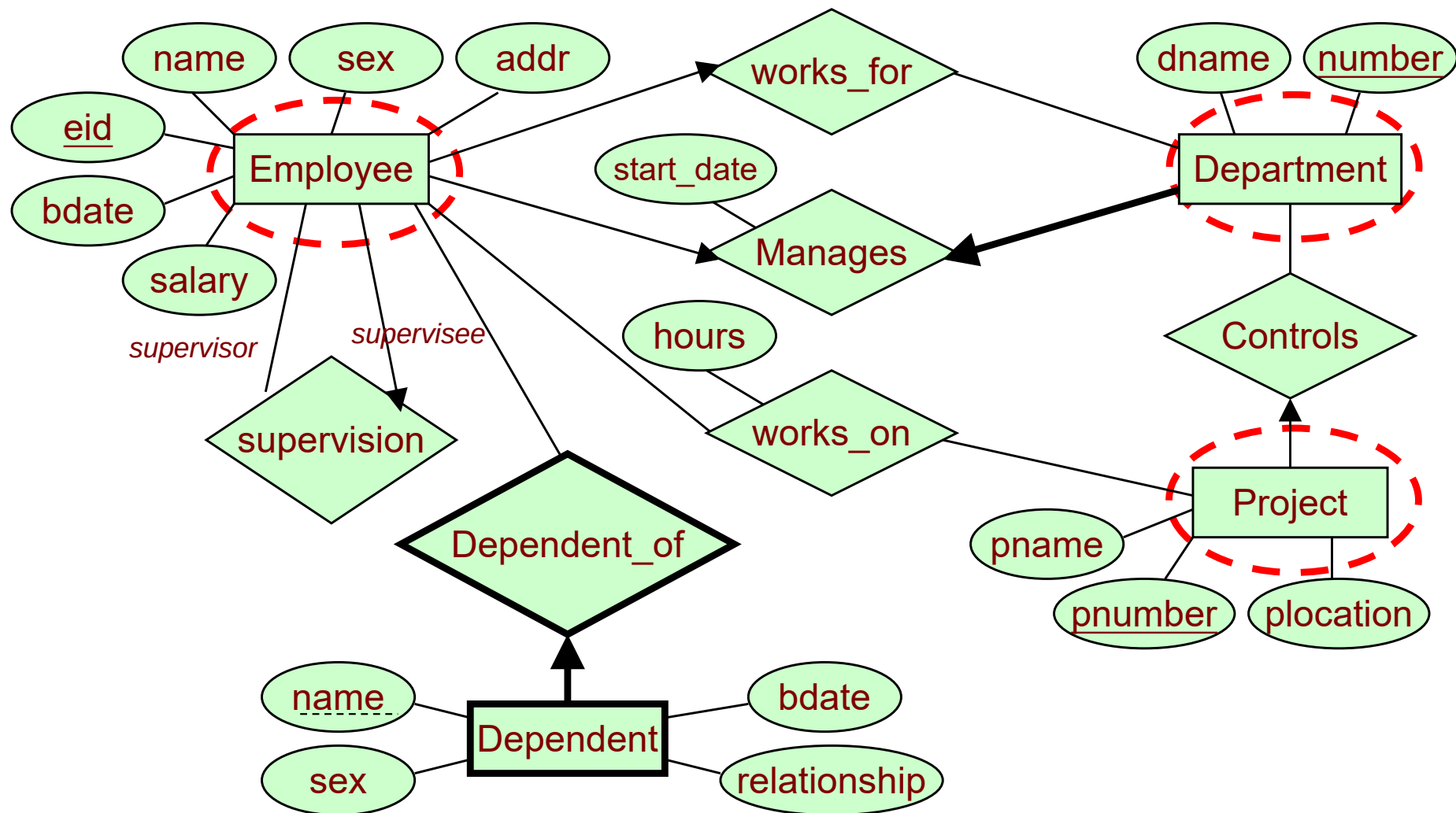
- From 1-to-1, 1-to-many (many-to-1) to many-to-many
- From binary relationship to non-binary relationship

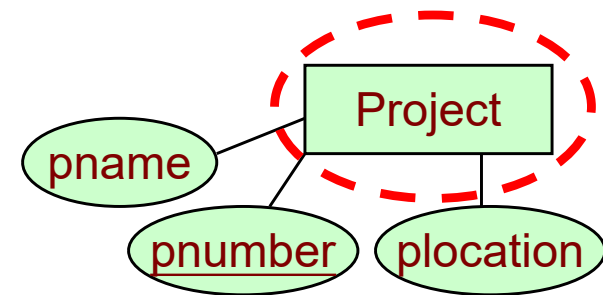
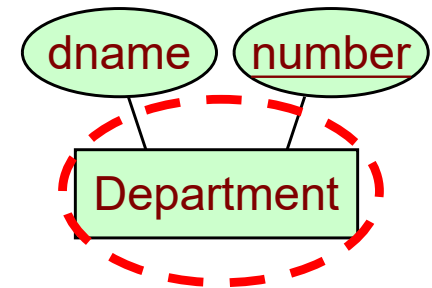
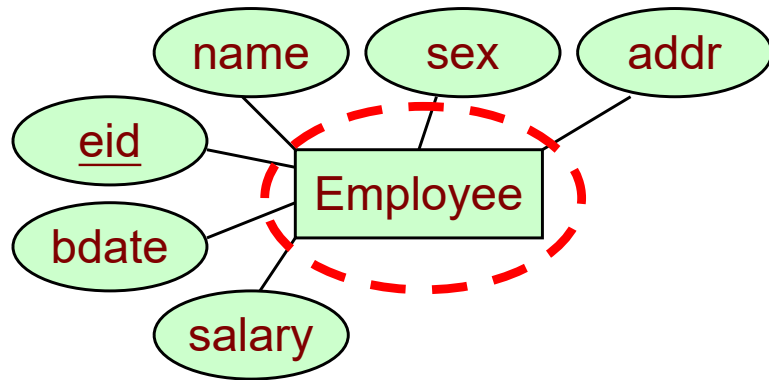
Steps

- ER-to-Relational Mapping Algorithm
 - **Step 1:** Mapping of Regular Entity Types
 - **Step 2:** Mapping of Weak Entity Types
 - **Step 3:** Mapping of Binary 1:1 Relationship Types
 - **Step 4:** Mapping of Binary 1:N Relationship Types.
 - **Step 5:** Mapping of Binary M:N Relationship Types.
 - **Step 6:** Mapping of N-ary Relationship Types.

Step 1 (Strong Entity)

- For each strong entity set E in the ER schema,
 - create a relation schema R that includes all the attributes of E
 - choose one set of key attributes of E as a primary key for R
 - if the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R





Step 1 (Strong Entity)

- Example
- We create the relation schemas EMPLOYEE, DEPARTMENT and PROJECT

EMPLOYEE

name	<u>eid</u>	bdate	addr	sex	salary
------	------------	-------	------	-----	--------

DEPARTMENT

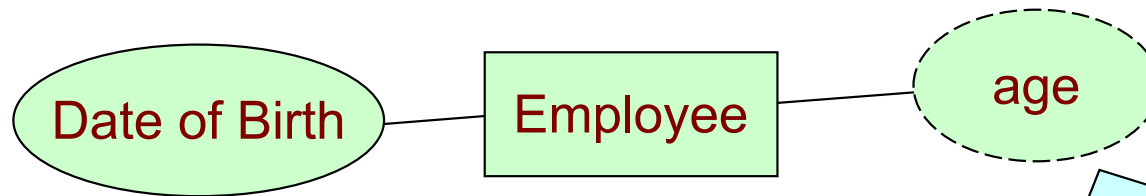
dname	<u>dnumber</u>
-------	----------------

PROJECT

pname	<u>pnumber</u>	plocation
-------	----------------	-----------

Step 1 (Strong Entity)

- If there is a derived attribute, what should we do?



We have two choices.

Choice 1: Include this derived attribute

Adv: We can directly obtain the value of the derived attribute

Disadv: We may encounter some data inconsistencies

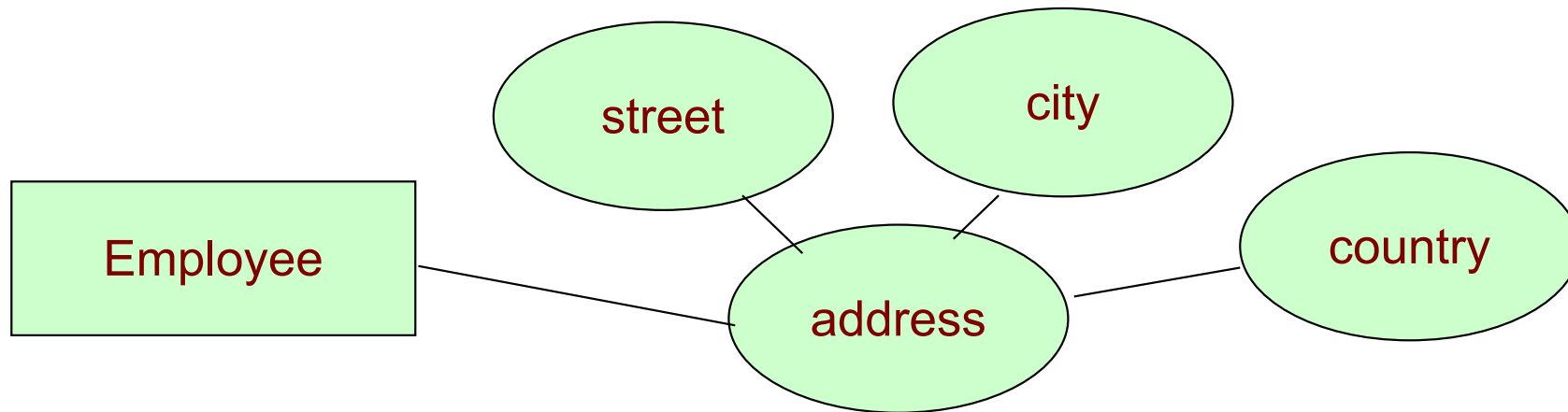
Choice 2: NOT include this derived attribute

Adv: We can avoid data inconsistency

Disadv: We need to perform some operations to obtain the value of the derived attribute

Step 1 (Strong Entity)

- If there is a composite attribute, what should we do?



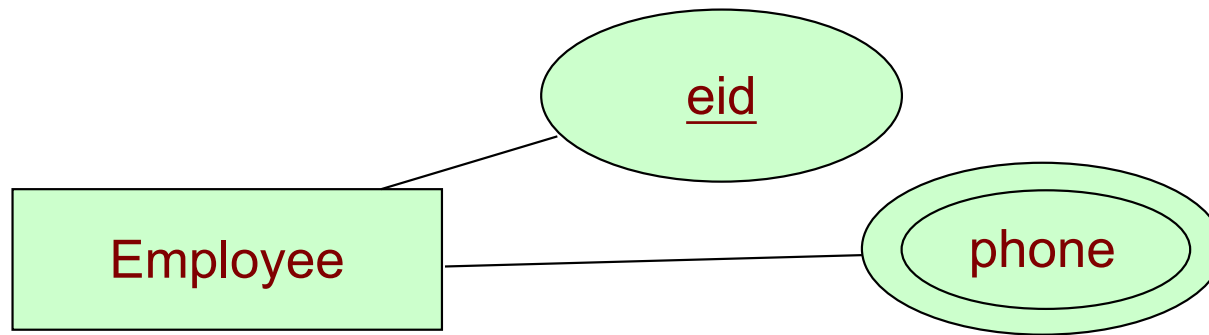
We have two choices.

Choice 1: Include the high-level attribute only (e.g., address)

Choice 2: Include all low-level attributes (e.g., street, city, country)

Step 1 (Strong Entity)

- If there is a multi-valued attribute, what should we do?



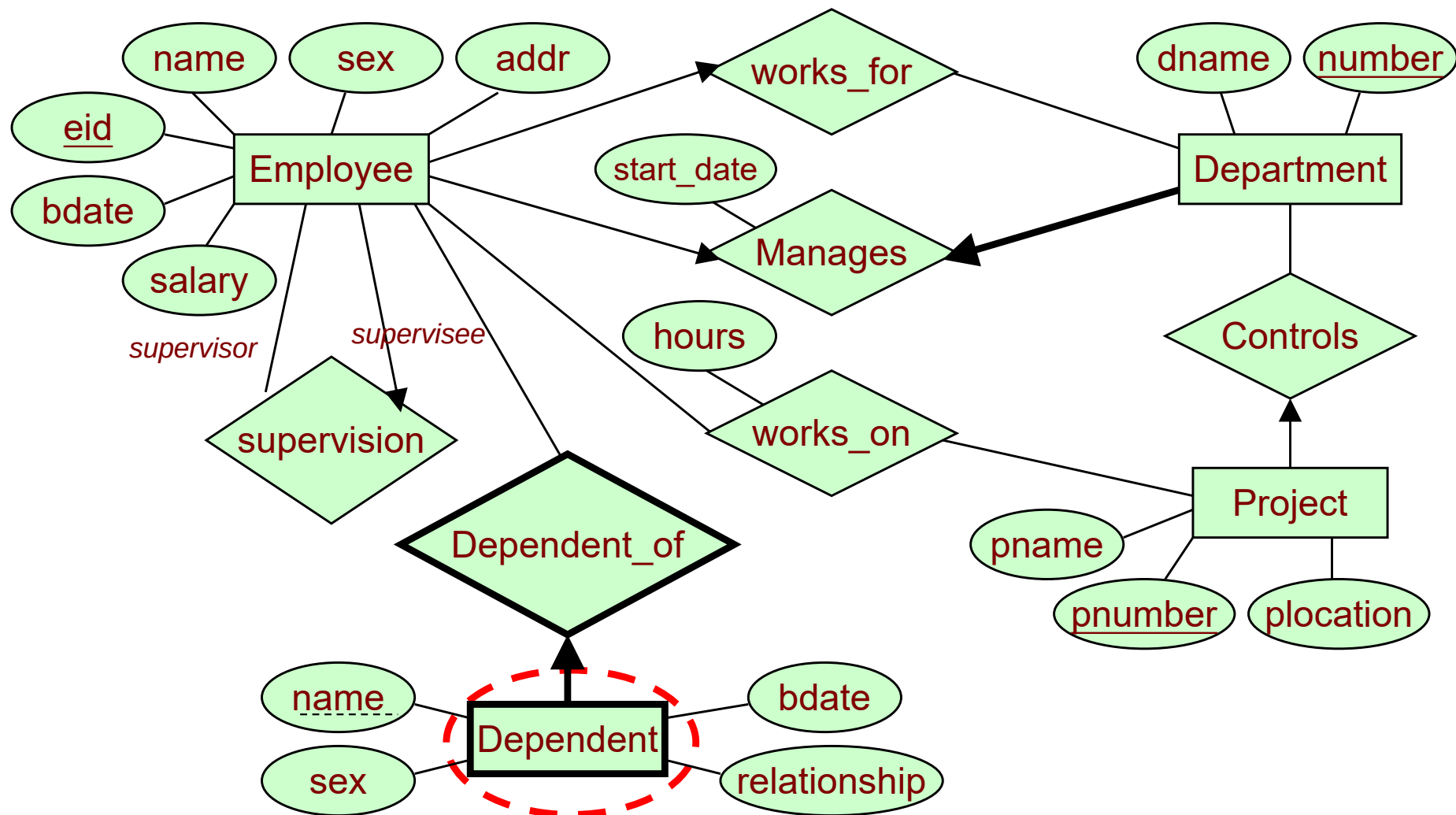
We have two choices.

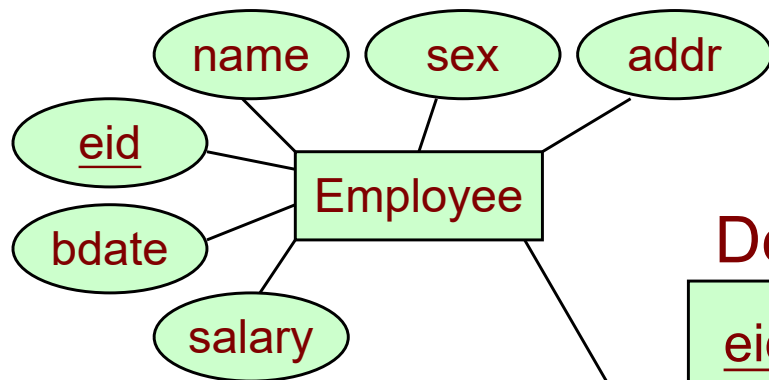
Choice 1: Include one attribute only (e.g., phone)

Choice 2: Create another table containing the primary key of the entity set and the multi-valued attribute. e.g., create a schema PhoneTable (eid, phone)

Step 2 (Weak Entity)

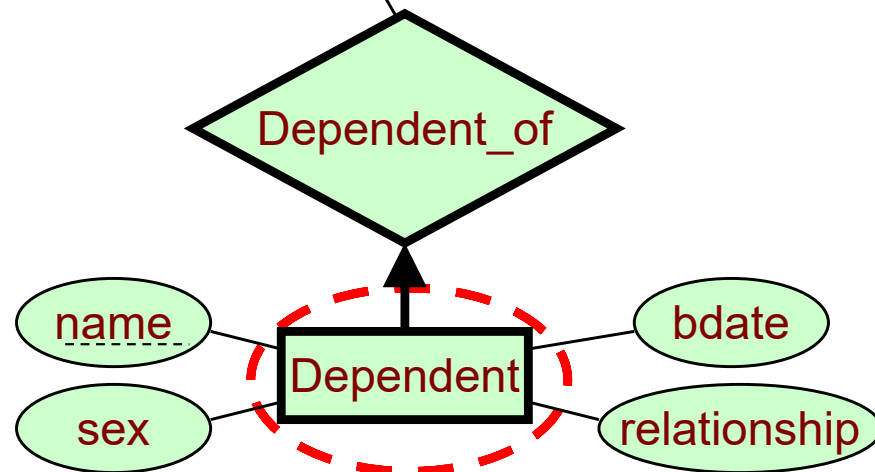
- For each **weak entity set** W in the ER model,
 - Create a relation schema R , and include all attributes
 - In addition, include the primary key(s) of the owner(s)
 - The primary key of R is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity set W





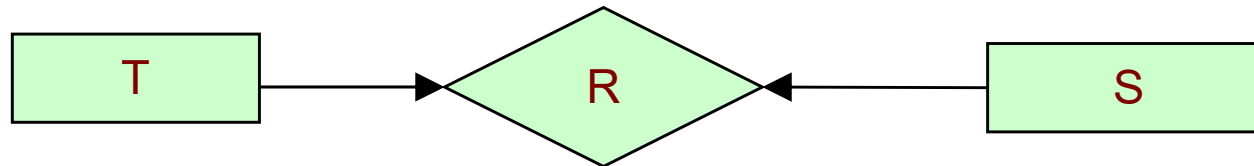
Dependent

<u>eid</u>	<u>dependent-name</u>	sex	bdate	relationship
------------	-----------------------	-----	-------	--------------

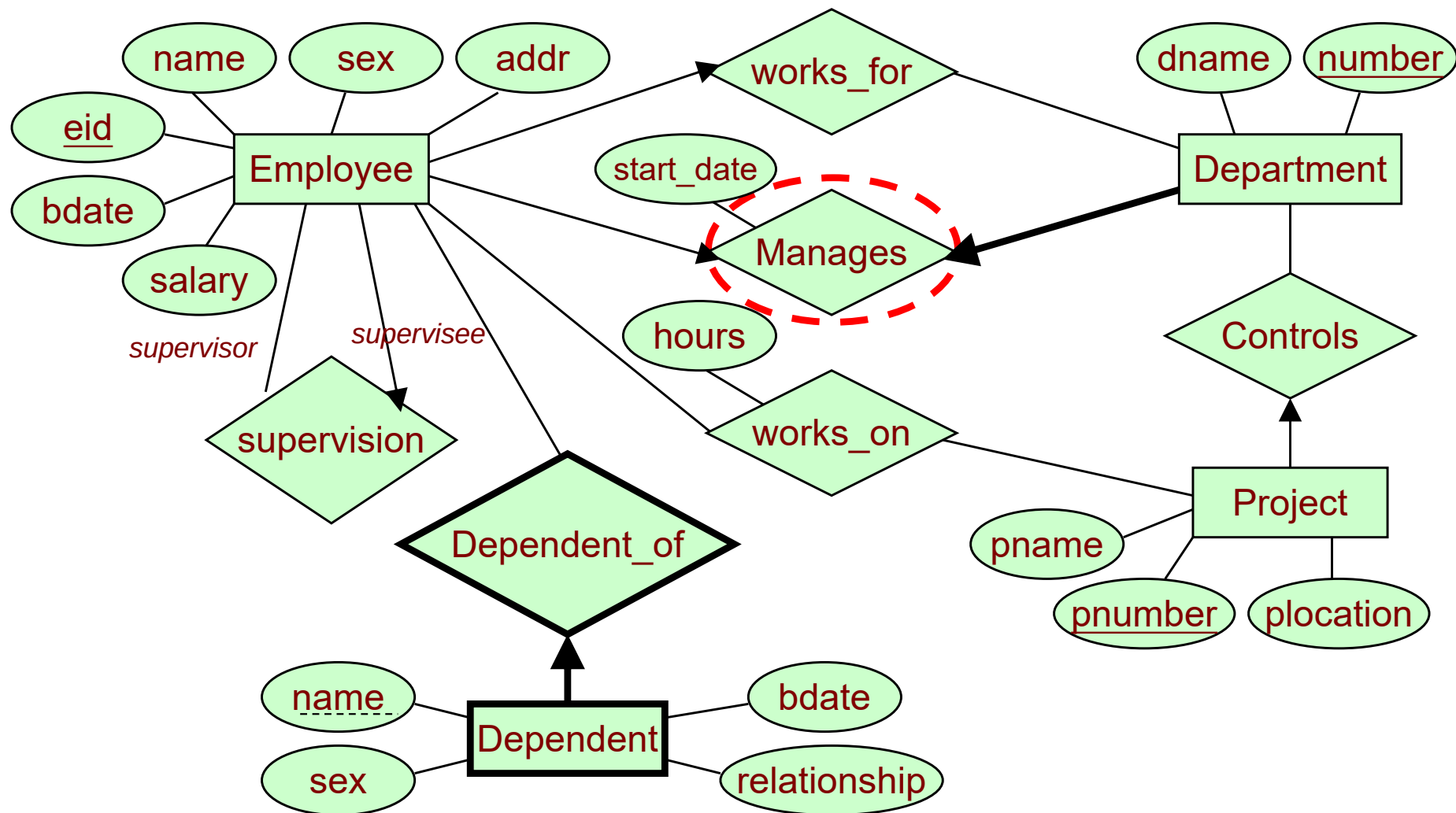


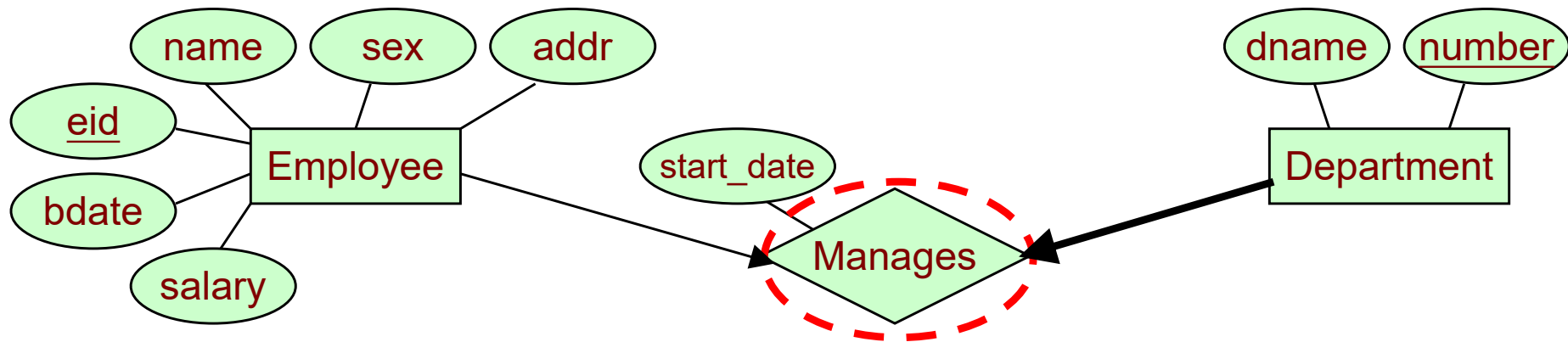
Step 3 (1-to-1 Relationship)

- For each binary one-to-one (1:1) relationship set R



- Choose one of the 2 relation schemas, say S,
 - get primary key of T,
 - include it as foreign keys in S
 - Include the attributes of the relationship set R as attributes of S
- Better if S has total participation in R



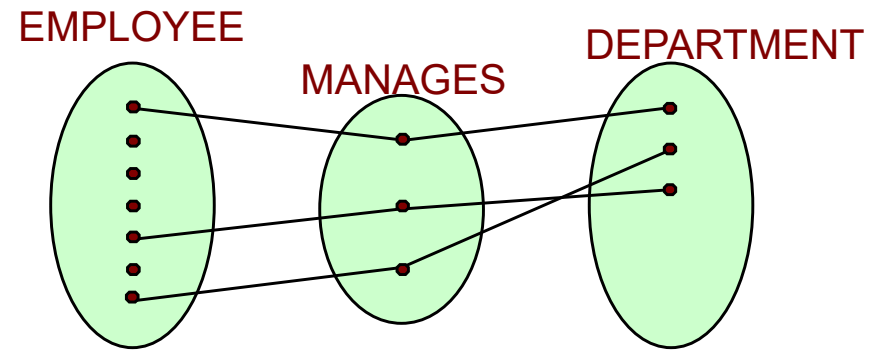


- We include the **primary key** of EMPLOYEE as foreign key in DEPARTMENT and rename it **mgr_id**
- We include the attribute **startdate** of MANAGES and rename it **mgr_start_date**

DEPARTMENT

dname	<u>dnumber</u>	mgr_id	mgr_start_date
-------	----------------	--------	----------------

Compare the following two choices
to include MANAGES:



Add information to EMPLOYEE

name	<u>id</u>	bdate	addr	sex	salary	dnum	sdate
Yeung	7	080370	...	F	20K	3	010100
Chan	3	031060	...	M	30K	4	020399
Wong	4	010280	...	F	10K	Null	Null
Cheung	8	220985	...	M	24K	Null	Null

dname	<u>dnumber</u>	mgr_id	mgr_start_date
Personnel	4	3	020399
Marketing	3	7	010100
...	...	...	...

Add to DEPARTMENT

- In the above, the NULL value is a special value meaning that the value is either unknown or not applicable
- Notice that an alternative mapping of a one-to-one relationship set is possible by merging the two entity sets and the relationship into a single relation
- This is appropriate when both participations are total

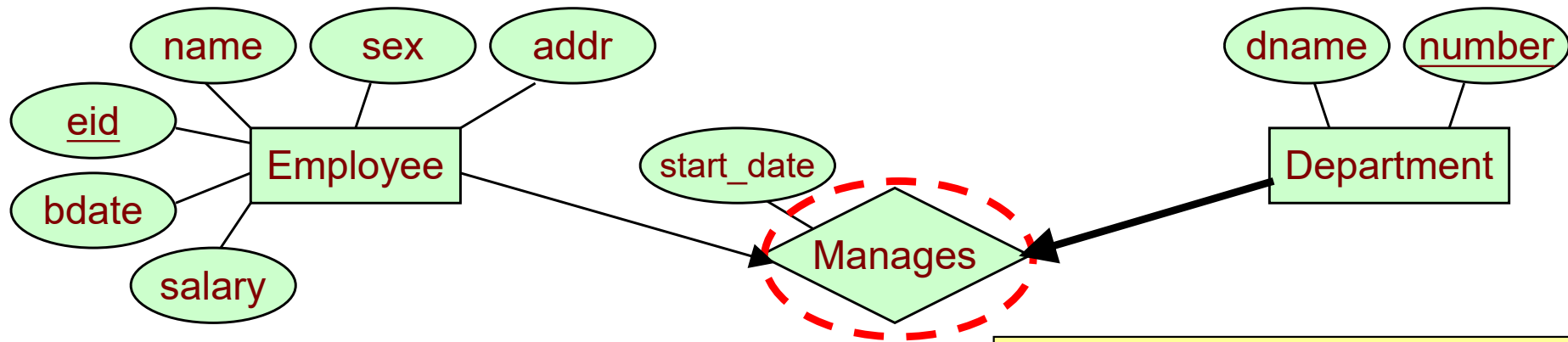
Mapping 1-to-1 Relationship:

Advantage:

The total number of relations remain unchanged

Disadvantage:

It may store NULL values if there is no total participation



Can we create a new relation

Manages (eid, number, start_date)

or

Manages (eid, number, start_date)

for this relationship?

Yes.

It can be used if there are only a few relationship instances

Advantage:

It can avoid storing NULL values if there is no total participation

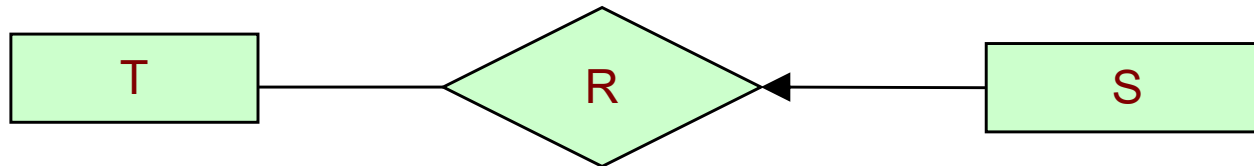
Disadvantage:

There is one additional relation

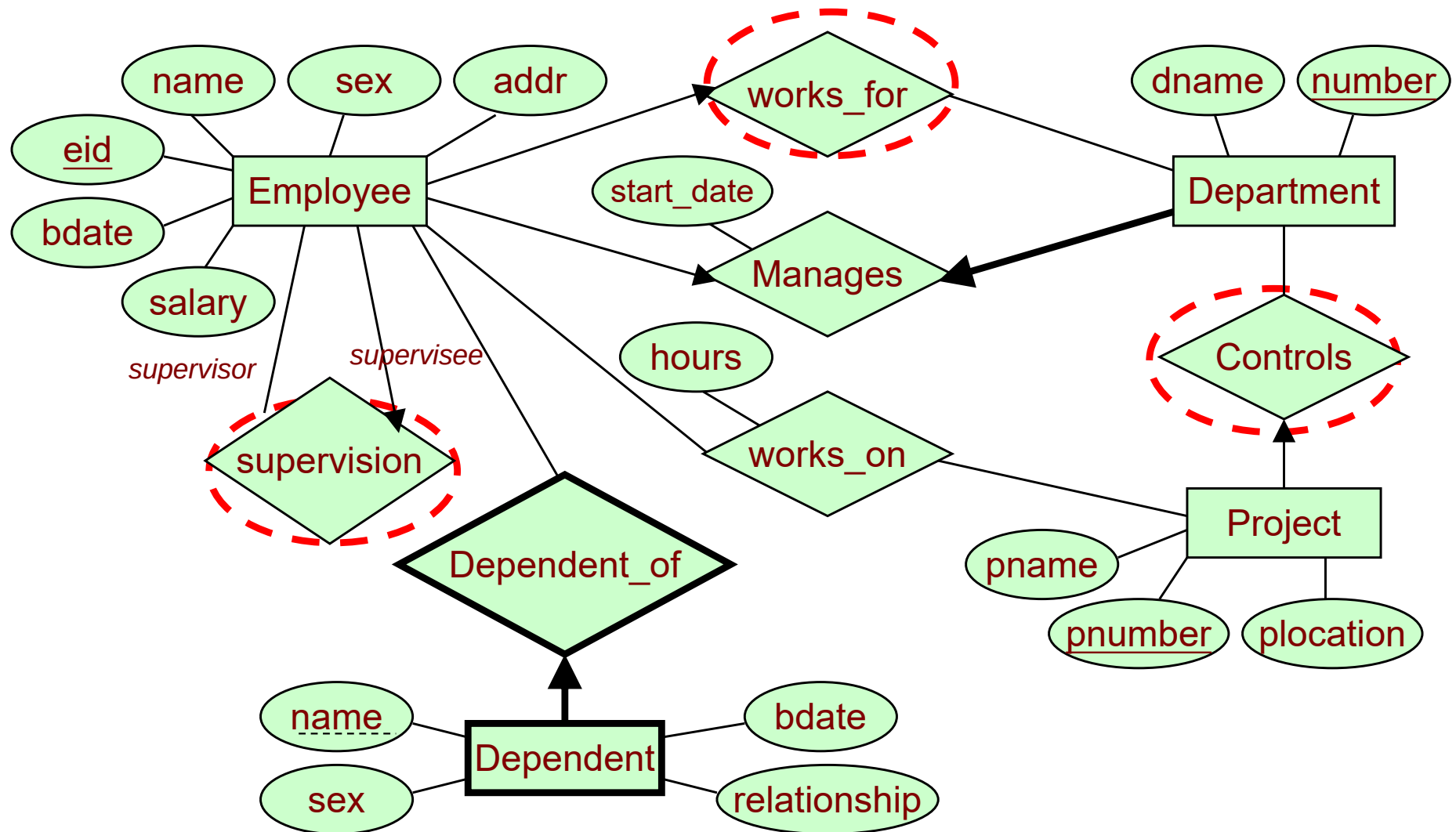
- There are three approaches for mapping 1-1 binary relationship:
 - 1. Foreign Key approach:** Choose one of the relations-say S-and include a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S
 - 2. Merged relation option:** An alternate mapping of a 1:1 relationship type is possible by merging the two entity types and the relationship into a single relation. This may be appropriate when both participations are total
 - 3. Cross-reference or relationship relation option:** The third alternative is to set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types

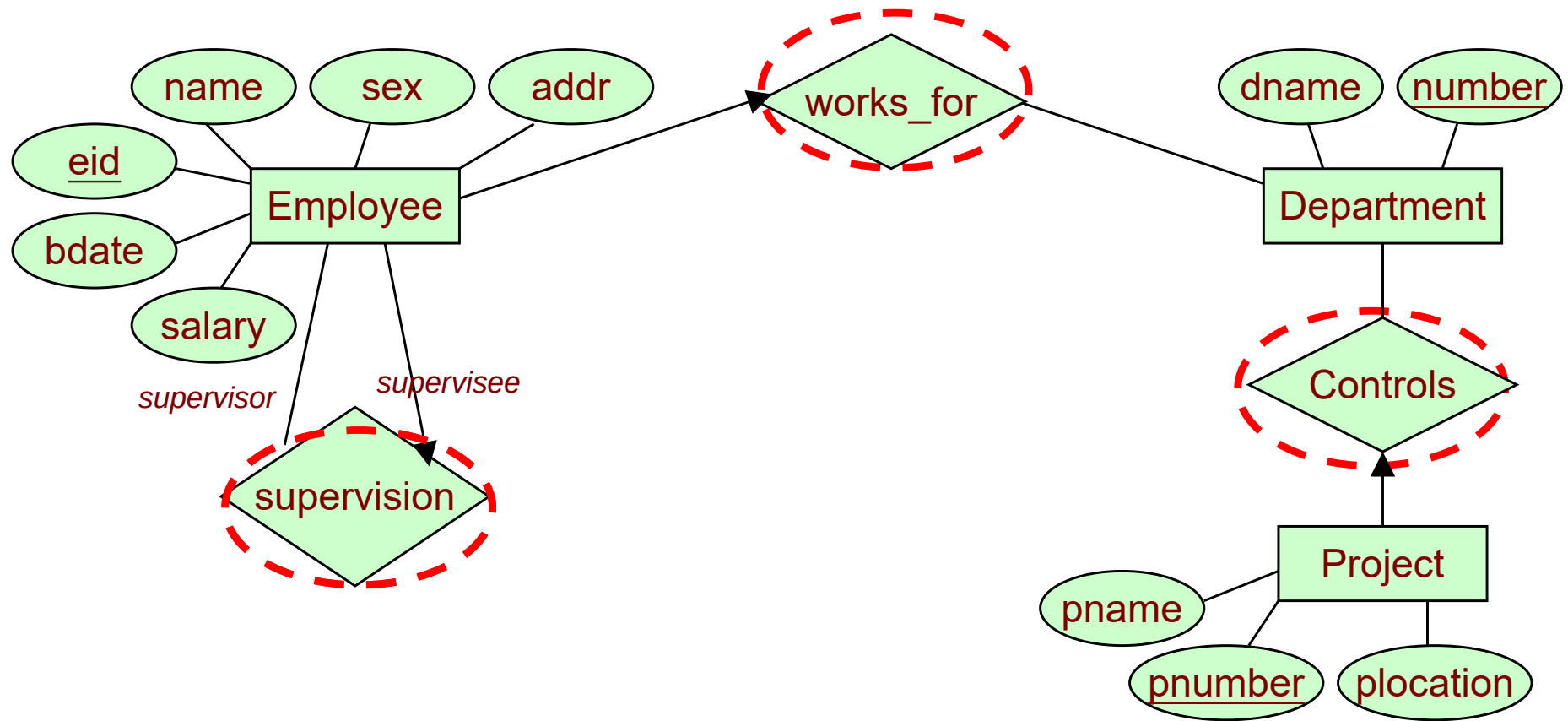
Step 4 (1-to-many Relationship)

- For each **binary one-to-many** relationship set



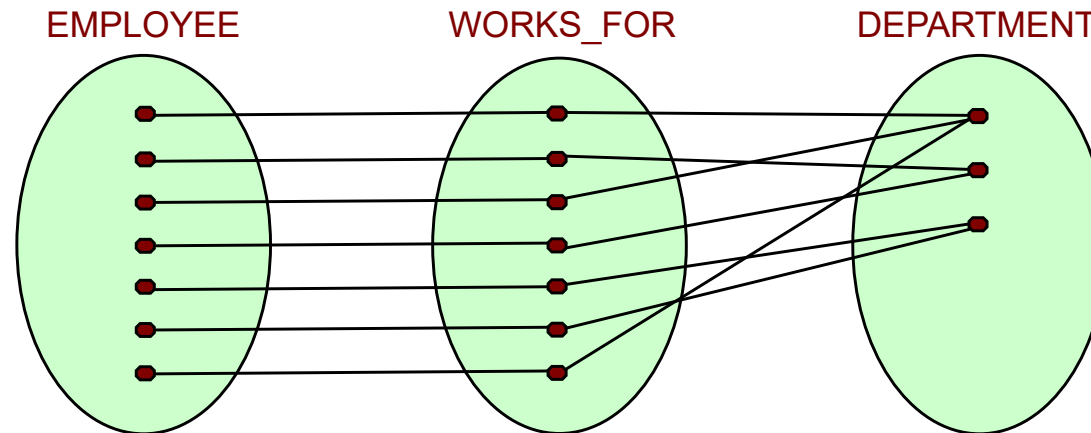
- Include as foreign key in S the primary key that represents the other entity set T participating in R
- Include any attributes of the one-to-many relationship set as attributes of S
- In other words, we always choose the many side as S





- The primary key dnumber of the DEPARTMENT relation schema is included as foreign key in the EMPLOYEE relation schema
- We rename it as dno (The renaming is not necessary but makes the name more meaningful.)

EMPLOYEE	name	<u>eid</u>	bdate	addr	sex	salary	dno



Compare the following 2 choices:

Add employees to department



DEPARTMENT

dname	<u>dnumber</u>	mgr_id	mgr_start_date	<u>eid</u>
Personnel	4	3	020399	3
Personnel	4	3	020399	11
Personnel	4	3	020399	12
Personnel	4	3	020399	13
Marketing	3	7	010100	21
Marketing	3	7	010100	7
Marketing	3	7	010100	22
...	...	...	...	...

EMPLOYEE

name	<u>id</u>	bdate	addr	sex	salary	dno
Yeung	7	080370	...	F	20K	3
Chan	3	031060	...	M	30K	4
Wong	4	010280	...	F	10K	7
Cheung	8	220985	...	M	24K	1

Add department to employee

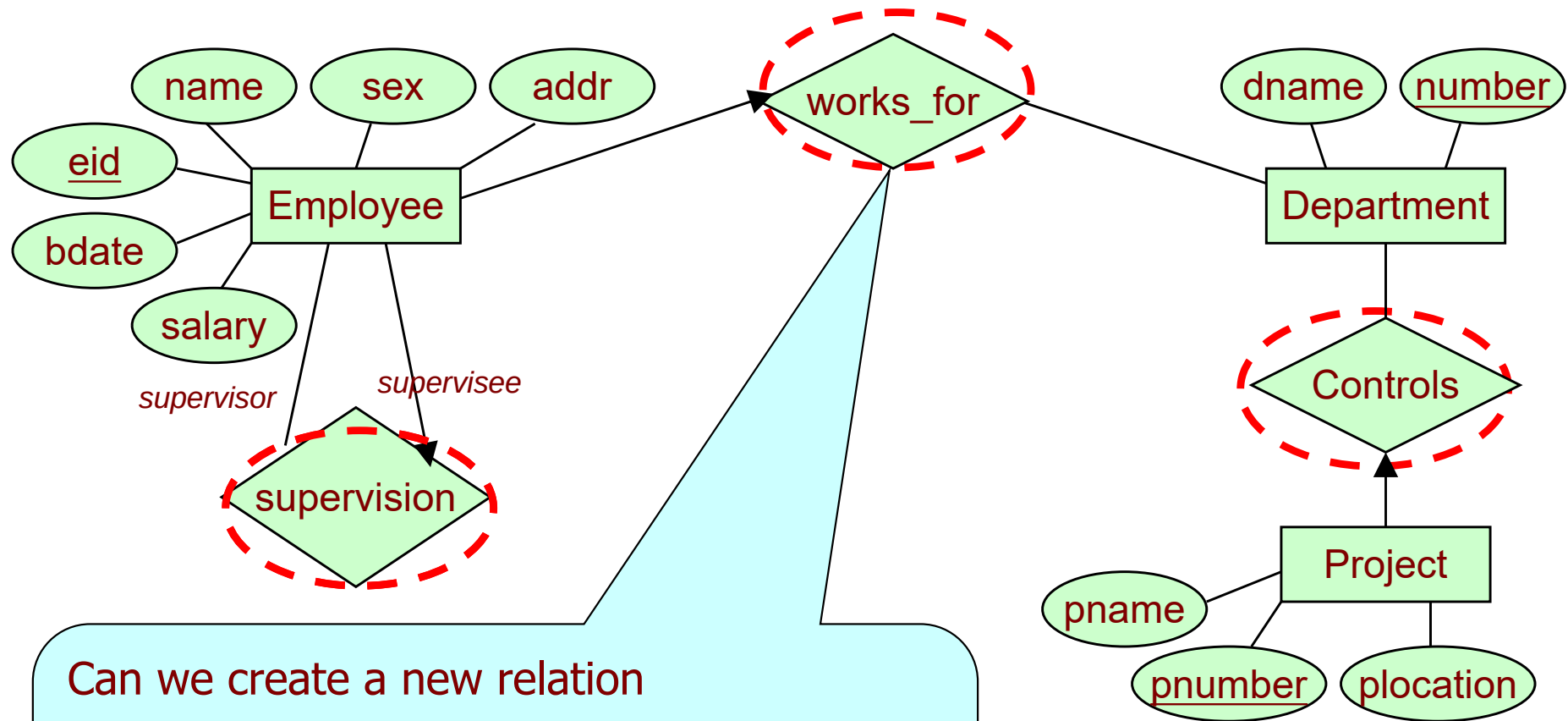


- For SUPERVISION,
 - include the primary key of the EMPLOYEE as foreign key in the EMPLOYEE, and call it super_id

EMPLOYEE	name	<u>eid</u>	bdate	addr	sex	salary	dno	super_id
----------	------	------------	-------	------	-----	--------	-----	----------

- For CONTROLS relationship,
 - include dnum as foreign key in PROJECT,
 - which references the primary key dnumber of DEPARTMENT

PROJECT	pname	<u>pnumber</u>	plocation	dnum
---------	-------	----------------	-----------	------



Can we create a new relation

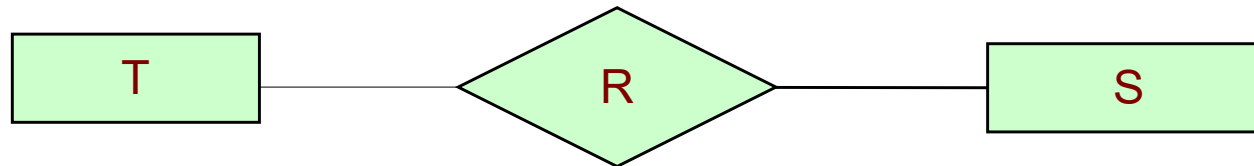
`works_for (eid, number)`

for this relationship?

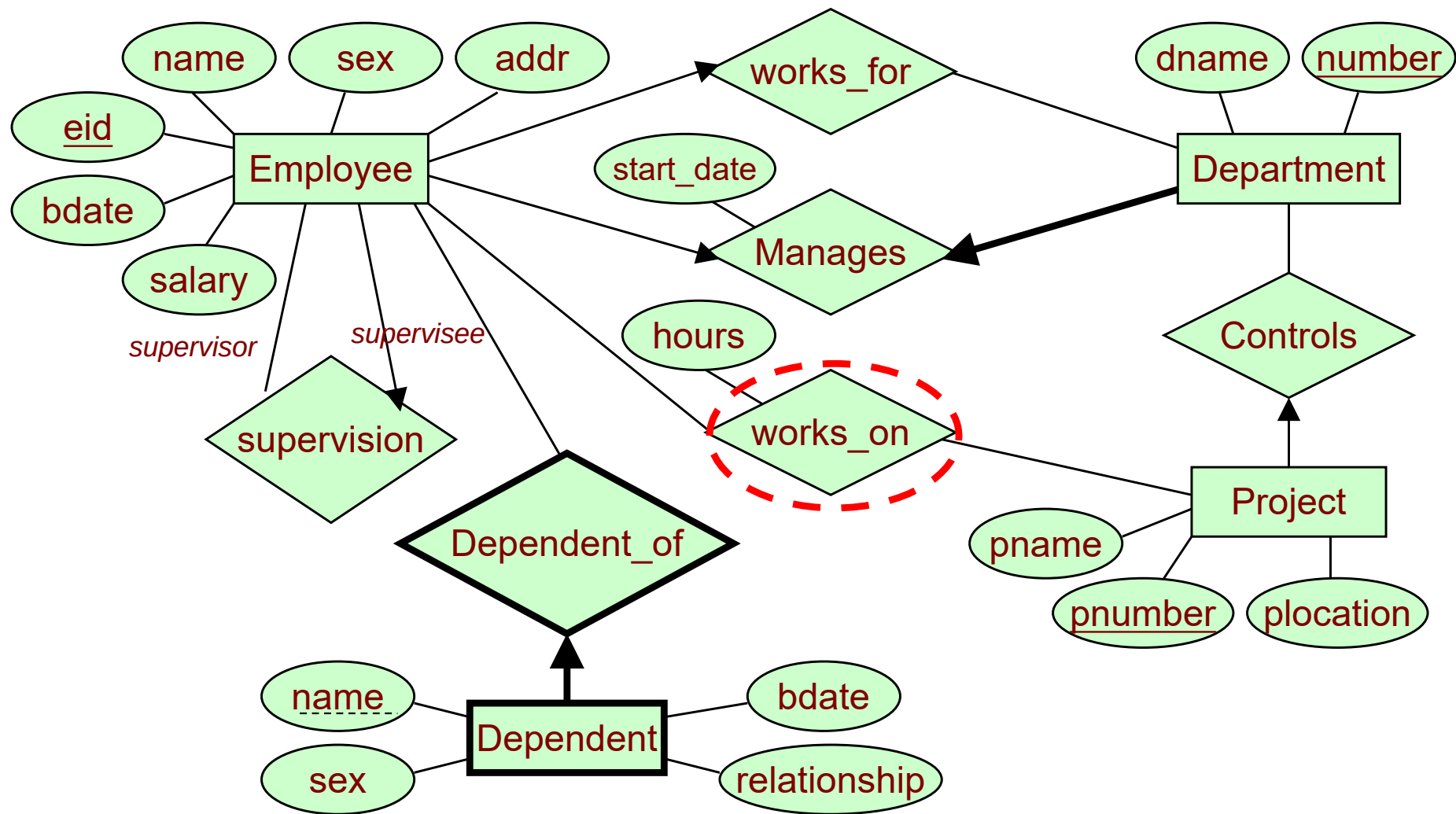
Yes.

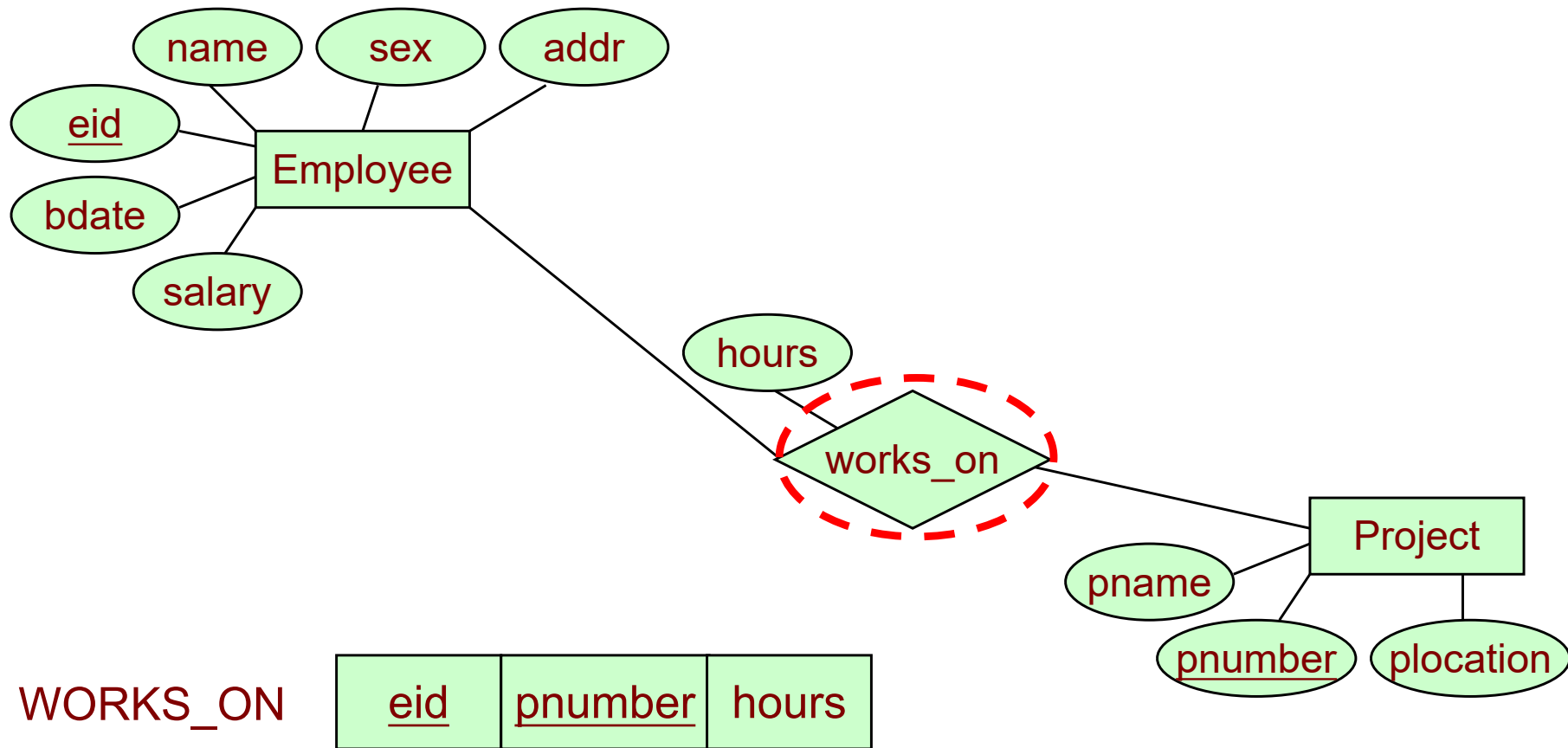
Step 5 (Many-to-many Relationship)

- For each binary many-to-many relationship set R



- Create a new relation schema P to represent R
- Include as foreign key attributes in P the primary keys of the relation schemas for the participating entity sets in R
- Their combination will form the primary key of P
- Also include attributes of the many-to-many relationship set as attributes of P





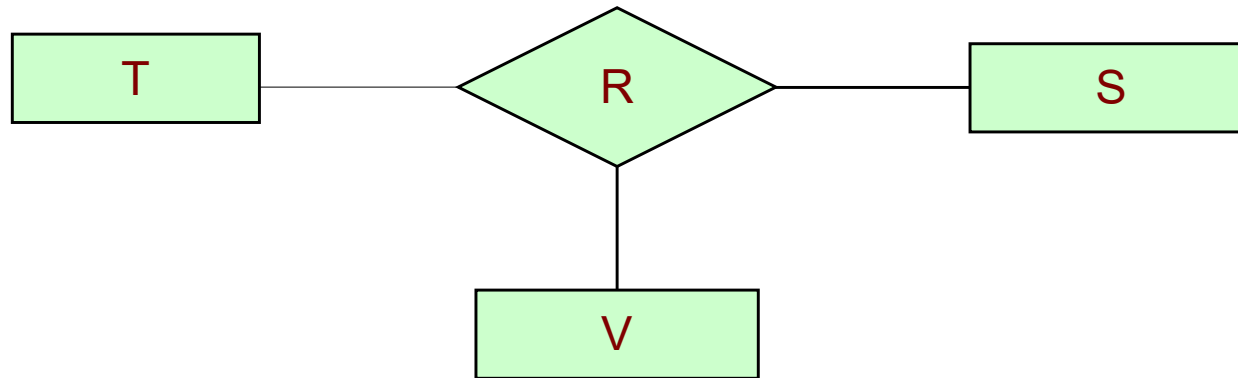
Map the many-to-many relationship sets

WORKS_ON by creating the relation schema WORKS_ON

Include the primary keys of PROJECT and EMPLOYEE as foreign keys

Step 6 (Non-binary Relationship)

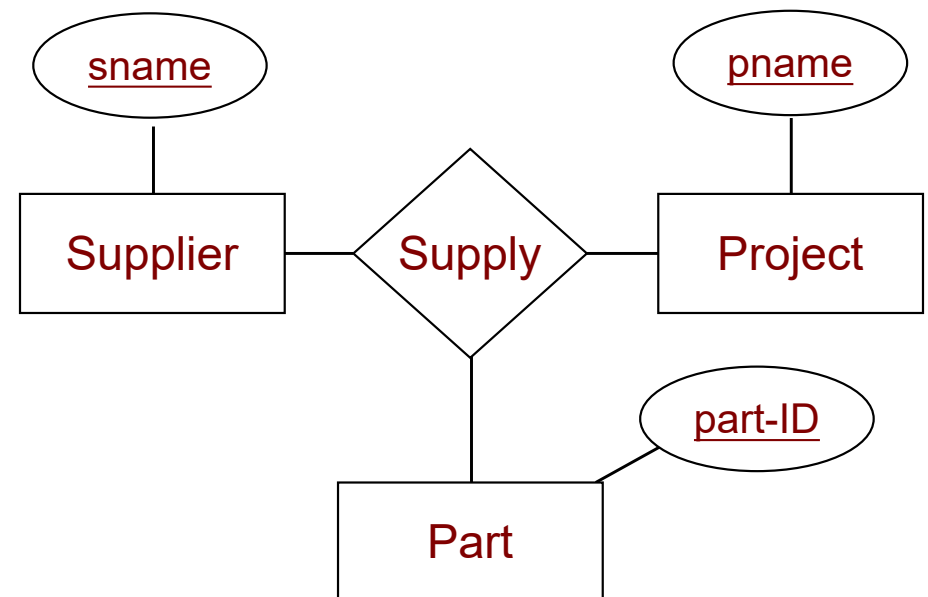
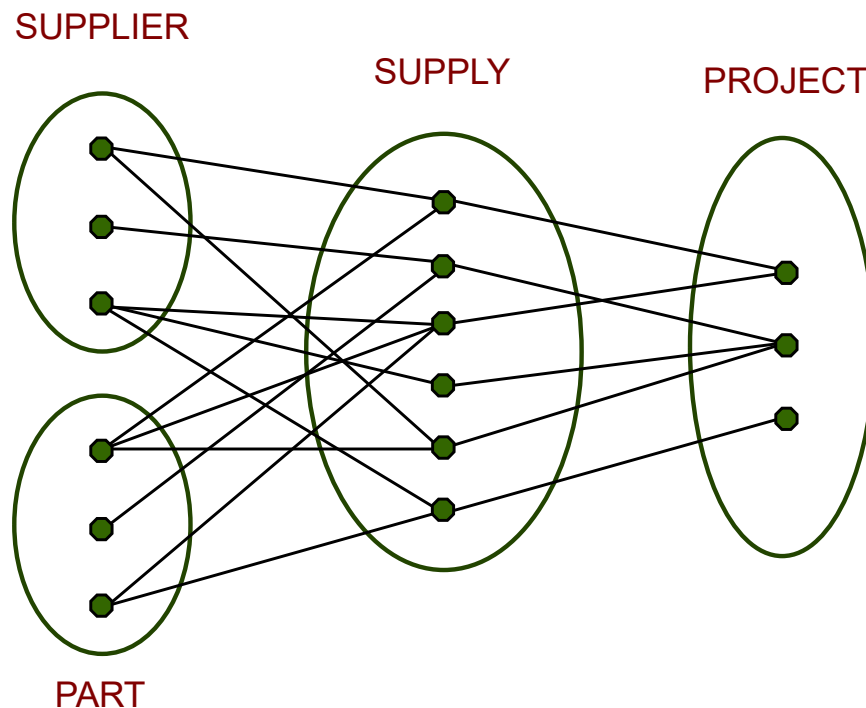
- For each non-binary relationship set



- Create a new relation schema P to represent R
- Include as foreign key attributes in P the primary keys of the participating entity sets
- Also include any attributes of the non-binary relationship set as attributes of P

Ternary Relationship Set

- Create table Supply,
- Attributes are sname, pname, part-ID,
- These also form the key



Supply (sname, pname, part-ID)

Resulting Relation Schemas:

DEPARTMENT	dname	<u>dnumber</u>	mgr_id	mgr_start_date
------------	-------	----------------	--------	----------------

EMPLOYEE	name	<u>eid</u>	bdate	addr	sex	salary	dno	super_id
----------	------	------------	-------	------	-----	--------	-----	----------

PROJECT	pname	<u>pnumber</u>	plocation	dnum
---------	-------	----------------	-----------	------

DEPENDENT	<u>eid</u>	<u>dependent-name</u>	sex	bdate	relationship
-----------	------------	-----------------------	-----	-------	--------------

WORKS_ON	<u>eid</u>	<u>pnumber</u>	hours
----------	------------	----------------	-------

Summary of Mapping Constructs

Correspondence between ER and Relational Models

ER Model

Entity type

1:1 or 1:N relationship type

M:N relationship type

n-ary relationship type

Simple attribute

Composite attribute

Multivalued attribute

Value set

Key attribute

Relational Model

“Entity” relation

Foreign key (or “relationship” relation)

“Relationship” relation and two foreign keys

“Relationship” relation and *n* foreign keys

Attribute

Set of simple component attributes

Relation and foreign key

Domain

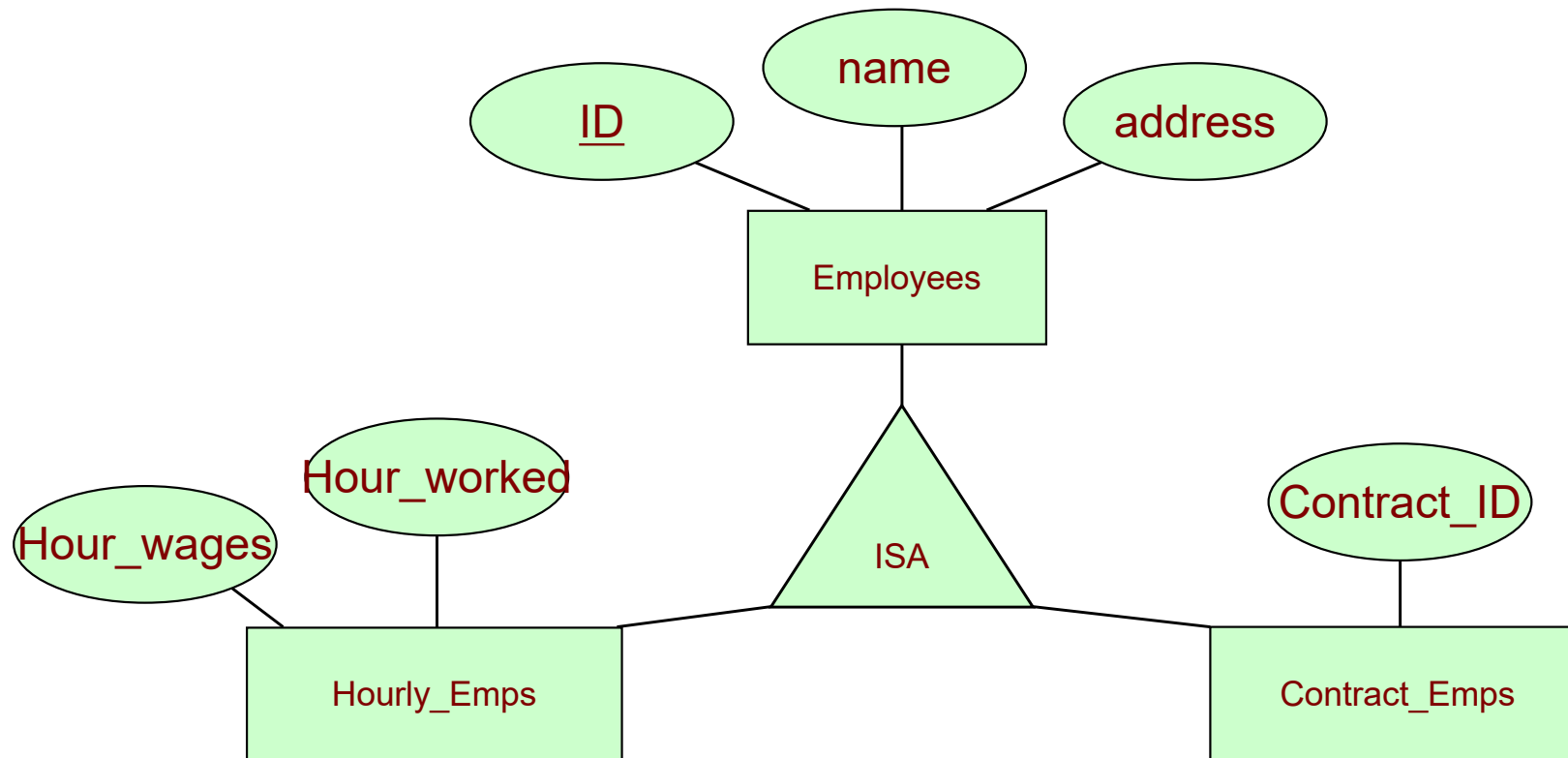
Primary key

Outline

- Relational Model
 - Relational Model Concepts
 - Relational Database Schemas
- **ER-to-Relational Mapping**
 - Translating traditional ER diagrams
 - **Translating Class Hierarchy**

Translating Class Hierarchy

- Consider the class hierarchy example



Translating Class Hierarchy

- Two possible ways:
 - 1. Map each of the entity sets Employees, Hourly-Emps, and Contract-Emps to a distinct relation

EMPLOYEE	name	<u>id</u>	address
----------	------	-----------	---------

HOURLY-EMP	<u>id</u>	hours_worked	hours_wages
------------	-----------	--------------	-------------

CONTRACT-EMP	<u>id</u>	contract-id
--------------	-----------	-------------

Translating Class Hierarchy

- Two possible ways:
 - 2. Create only two relations

HOURLY-EMP	<u>id</u>	name	address	hours_worked	hours_wages
CONTRACT-EMP	<u>id</u>	name	address	contract-id	

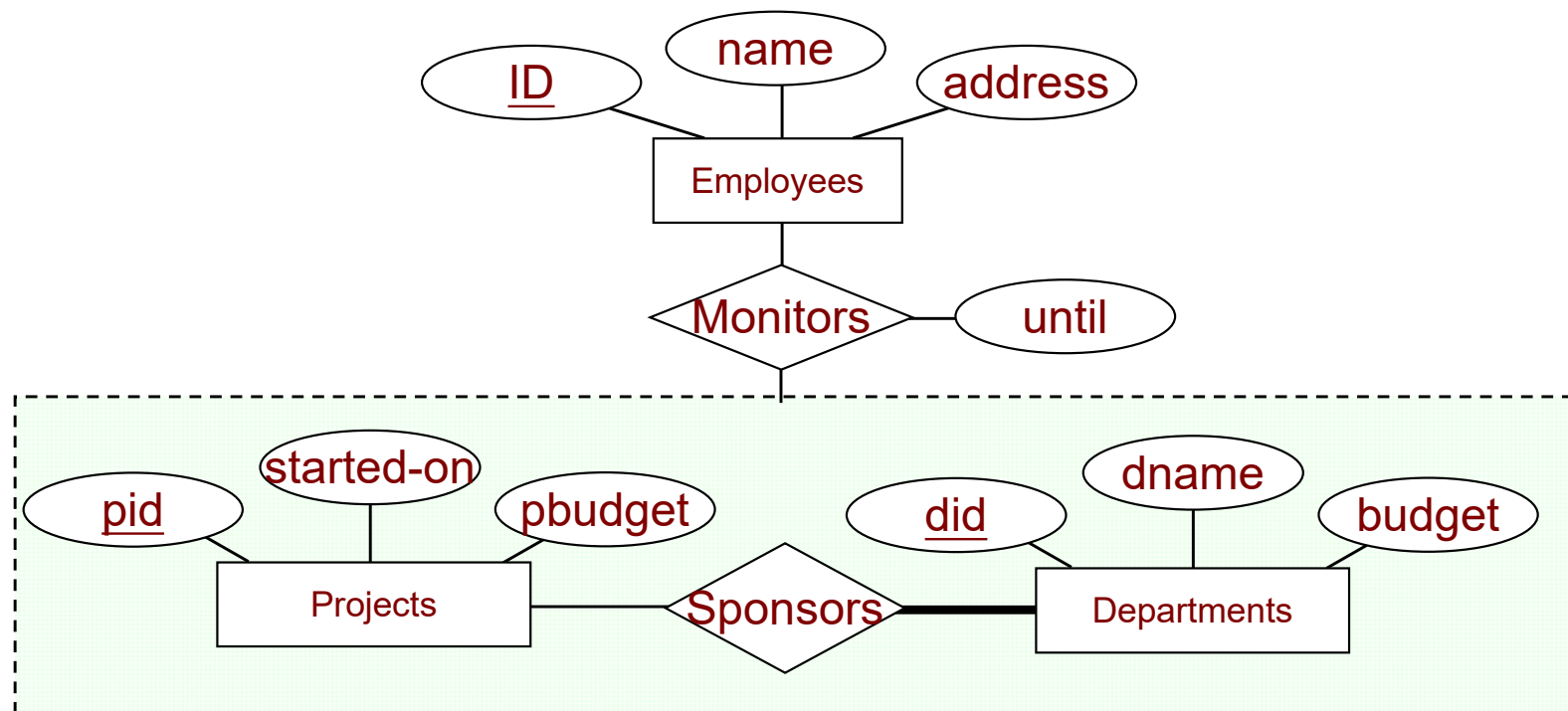
- This requires the covering constraint to hold. (i.e. Hourly_Emp and Contract_Emp COVER Employee)

Translating Class Hierarchy

- The first method is more general.
- Disadvantage:
 - an extra relation is needed
 - more operation may be necessary when we need to get the employee information (e.g. looking up two relations)
- The second method is not always possible
- Advantage:
 - information of each employee is more easily accessible (usually only one relation look up)
 - however, if an employee can be both hourly and contract based, then information of the employee will be stored twice

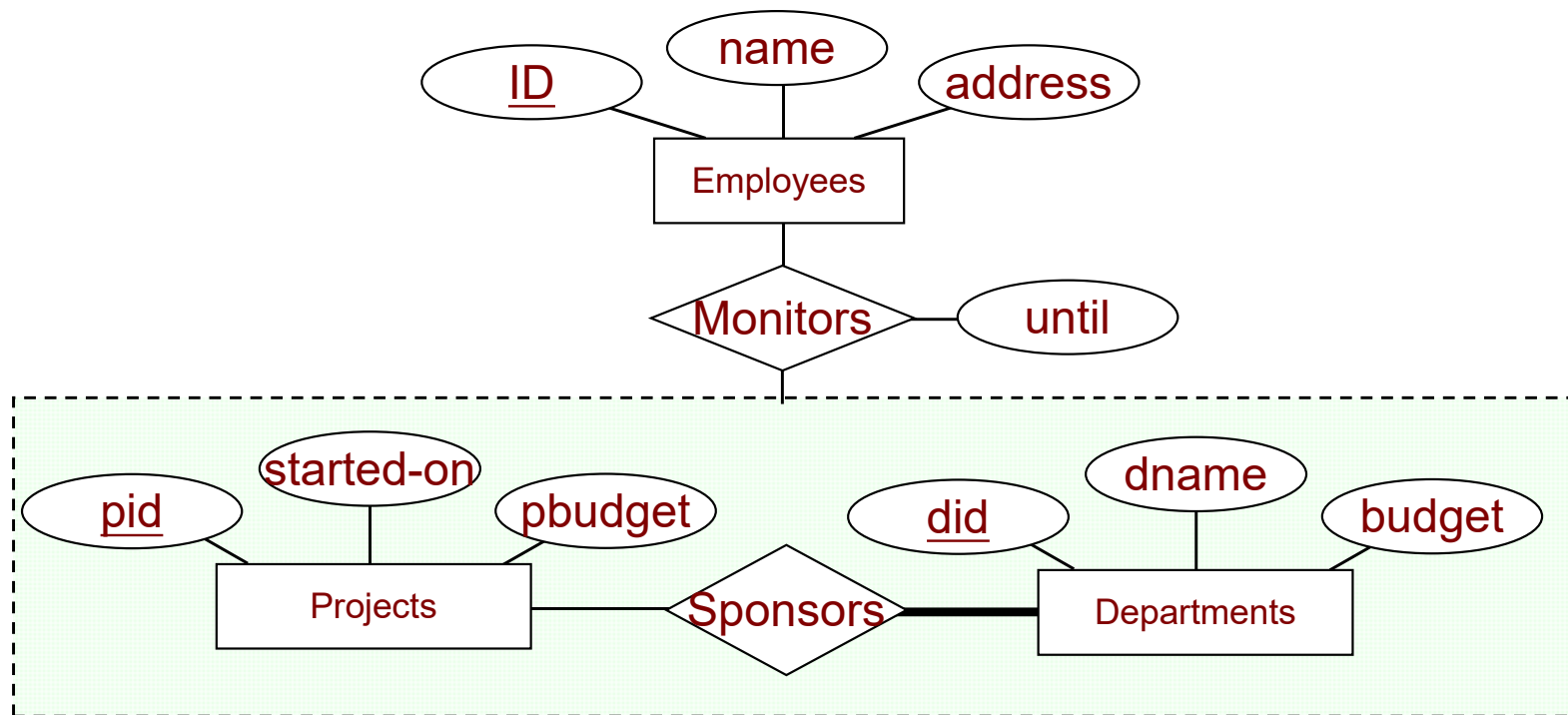
Translating Aggregation

- There is no real distinction between entities and relationships in the relational model
- Therefore the mapping of aggregation in the E-R model to the relational model is quite simple
- For example,

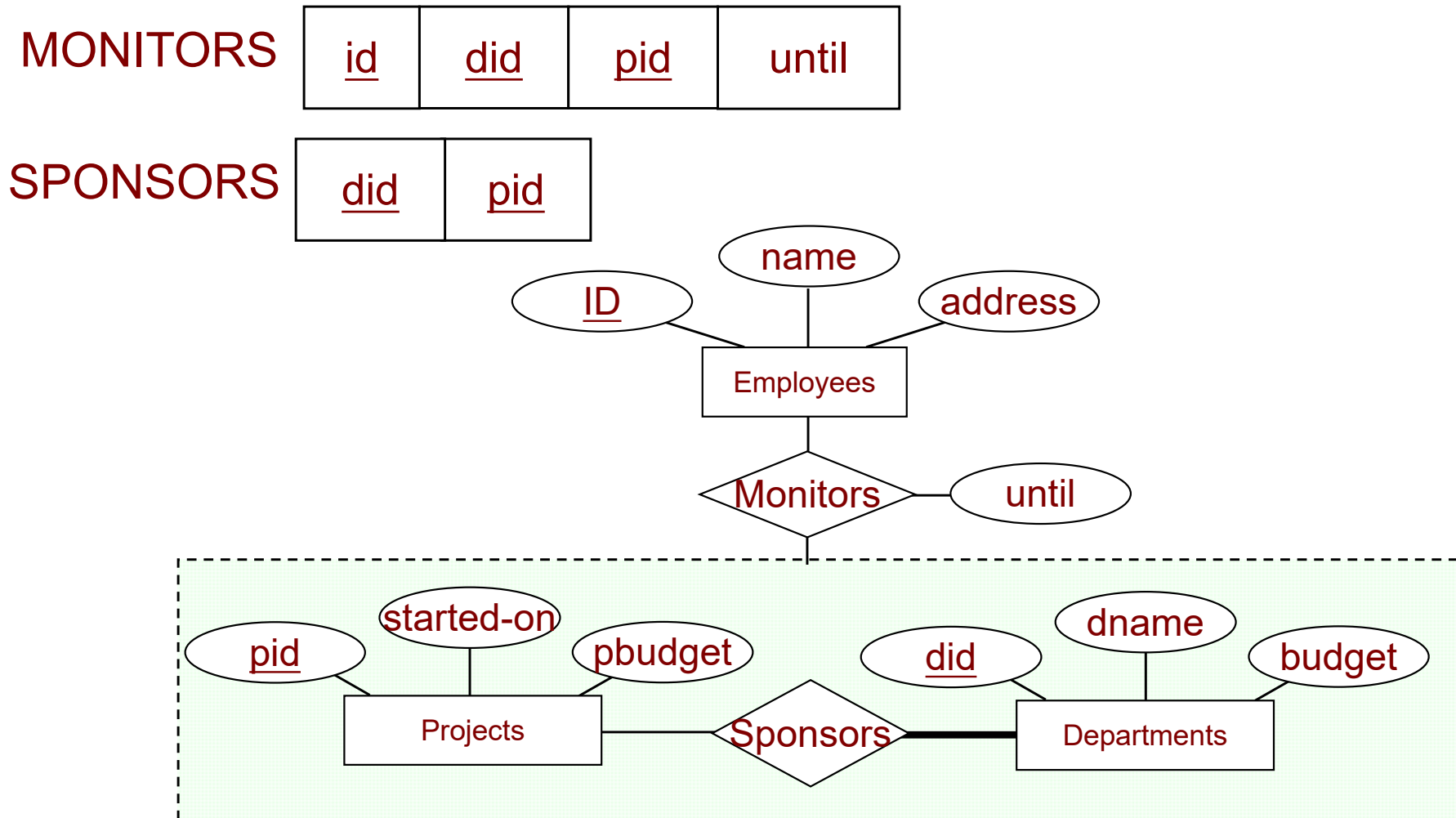


Translating Aggregation

- For example,
 - The key attributes of SPONSORS are: (did, pid)
 - (a sponsorship is determined by department id and project id)



- The key attributes of SPONSORS are: (did, pid)
- For the Monitors relationship we create a relation:

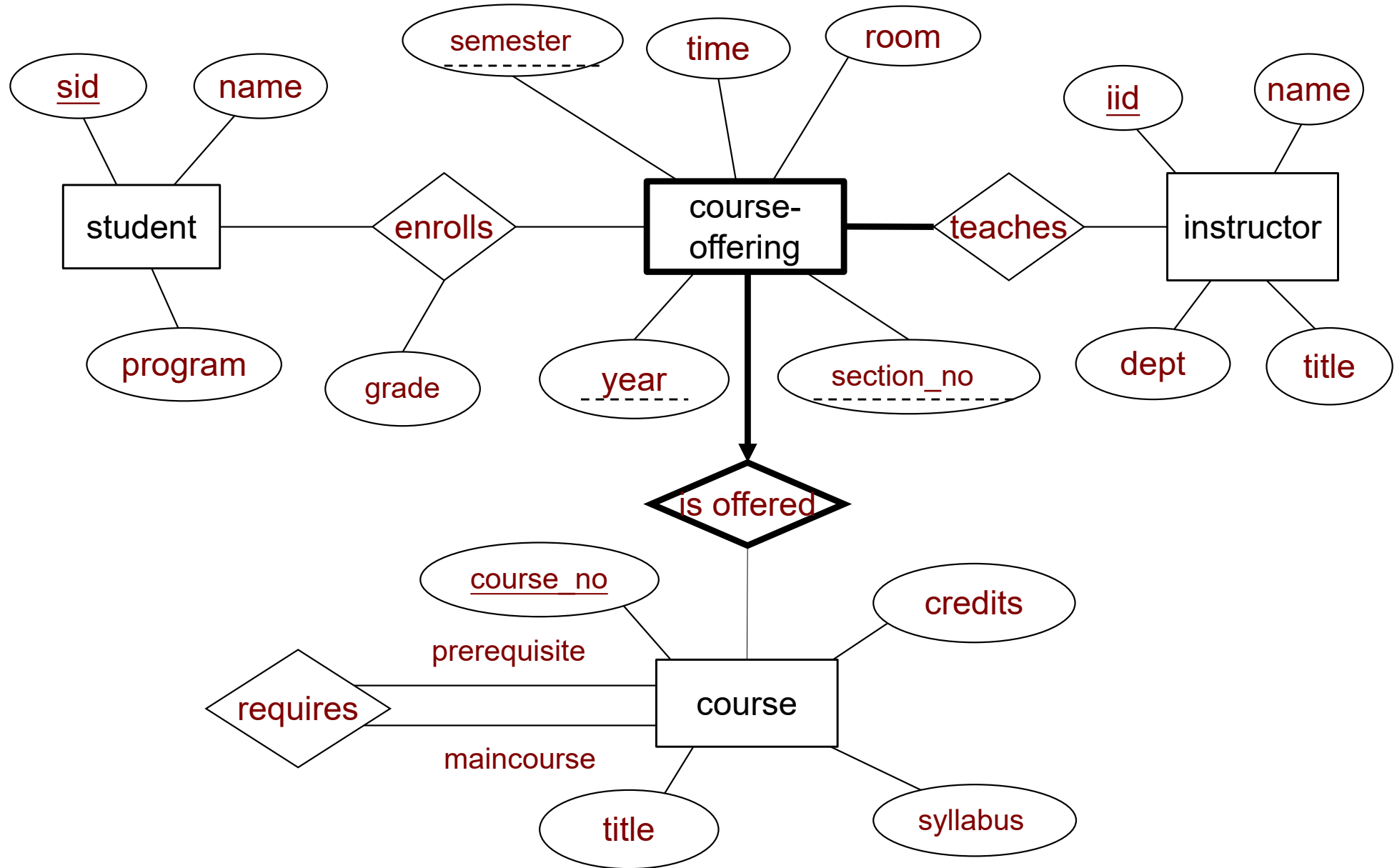


- Sponsors is not contained in Monitors above, if a sponsorship has no monitor, then it will not appear in Monitors

Summary

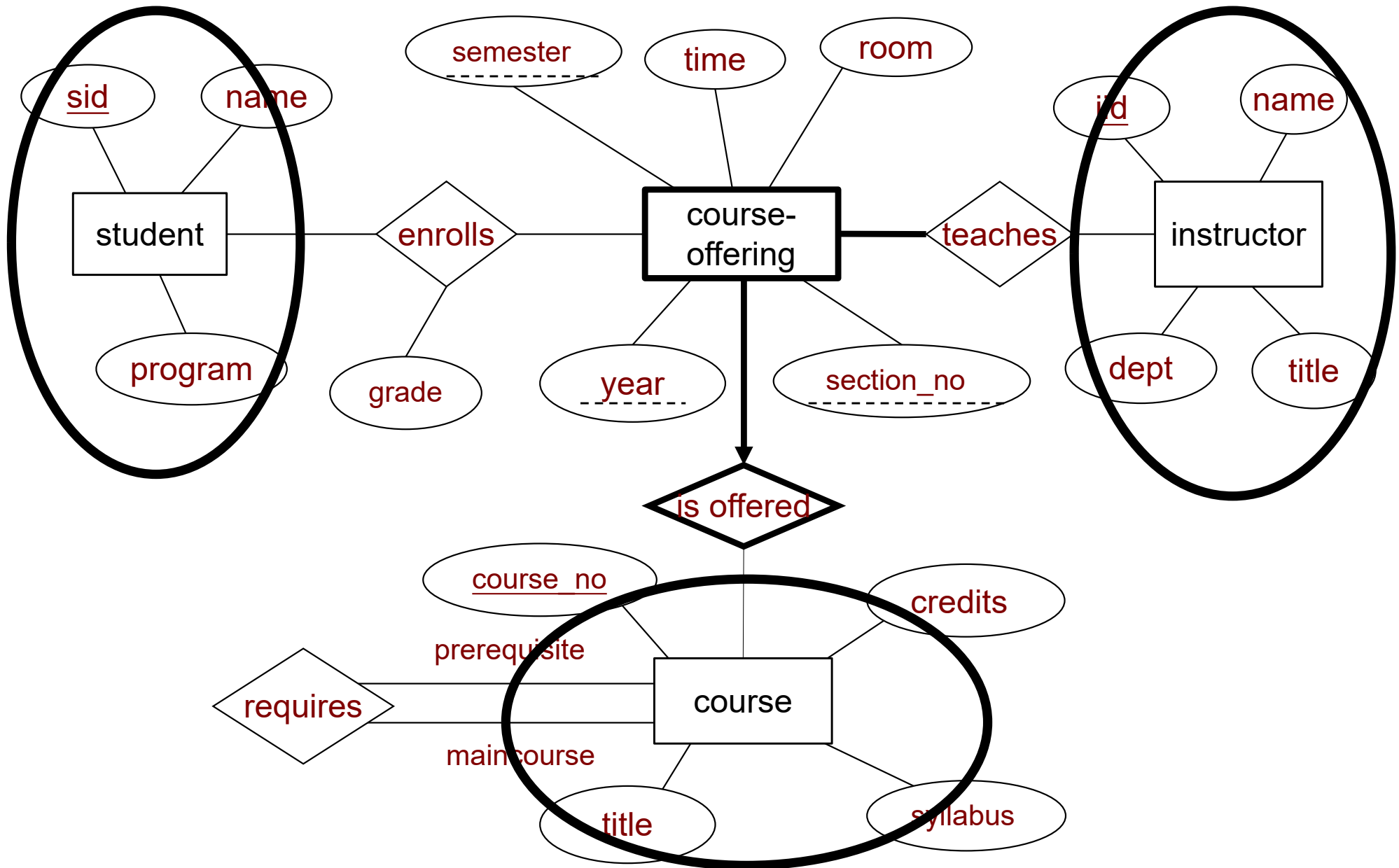
- What is a relation, relation schema
- How SQL creates tables (relations)
- Integrity Constraints - Primary Keys, Foreign Keys
Referential Integrity
- Translating ER to relational model
 - Translating class hierarchies, aggregations
 - Reduce data redundancy
 - Reduce the number of NULL values
 - Reduce the cost of lookup for the information

Exercise: Convert E-R Diagram into Tables



E-R diagram for a university.

String Entites



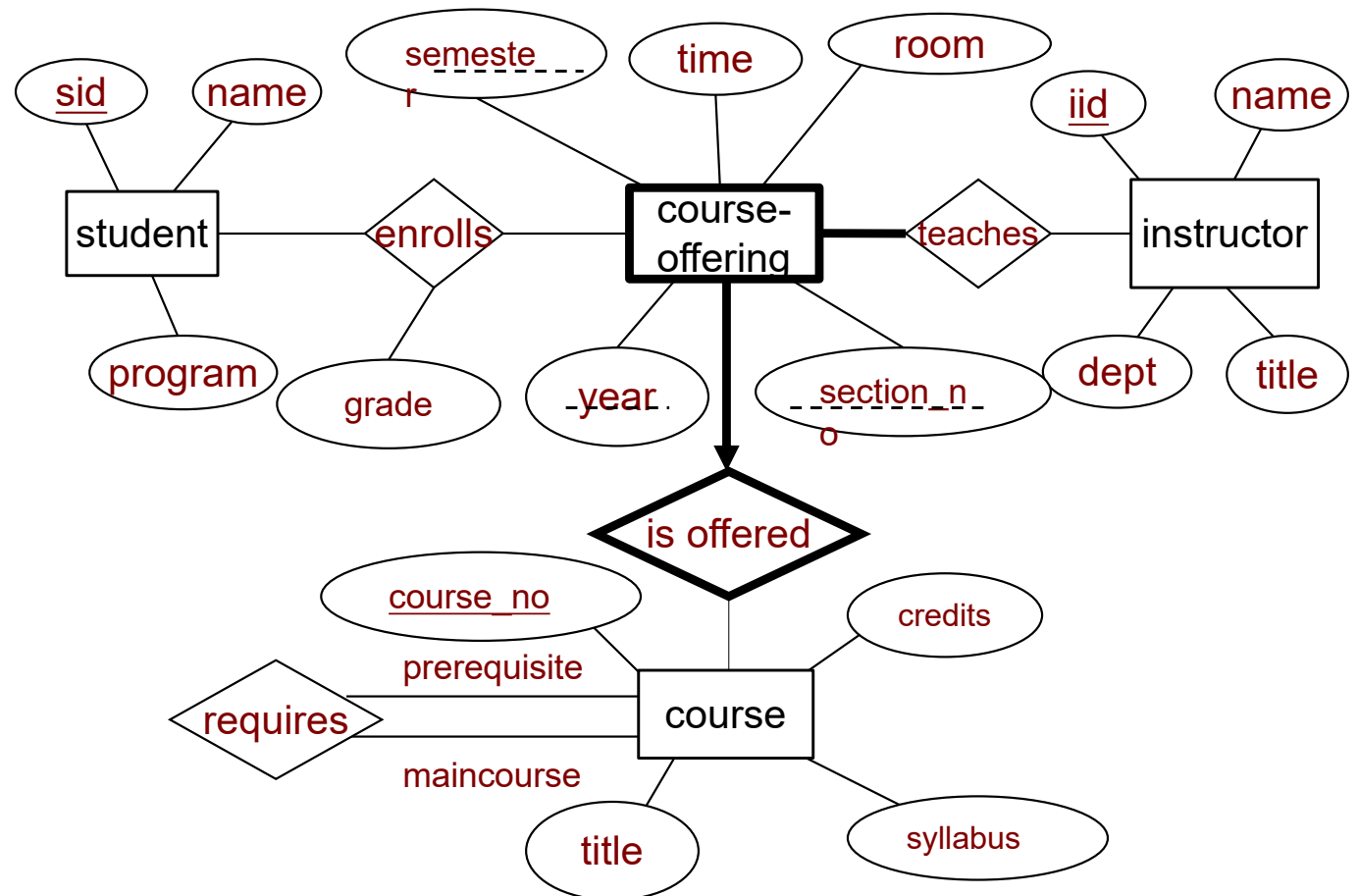
E-R diagram for a university.

Tables of Strong Entities

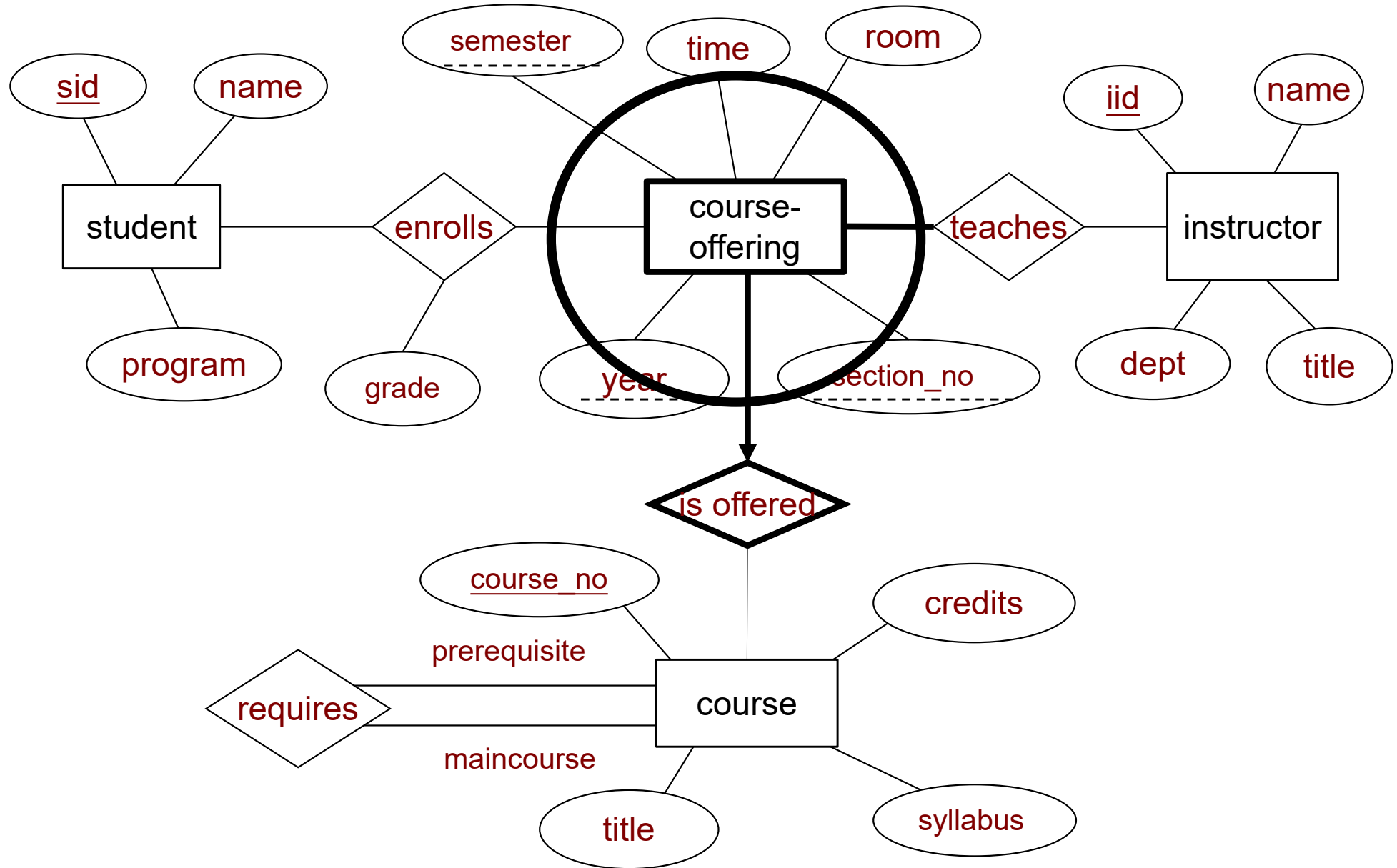
student (sid, name, program)

course (course_no, title, syllabus, credits)

instructor (iid, name, dept, title)



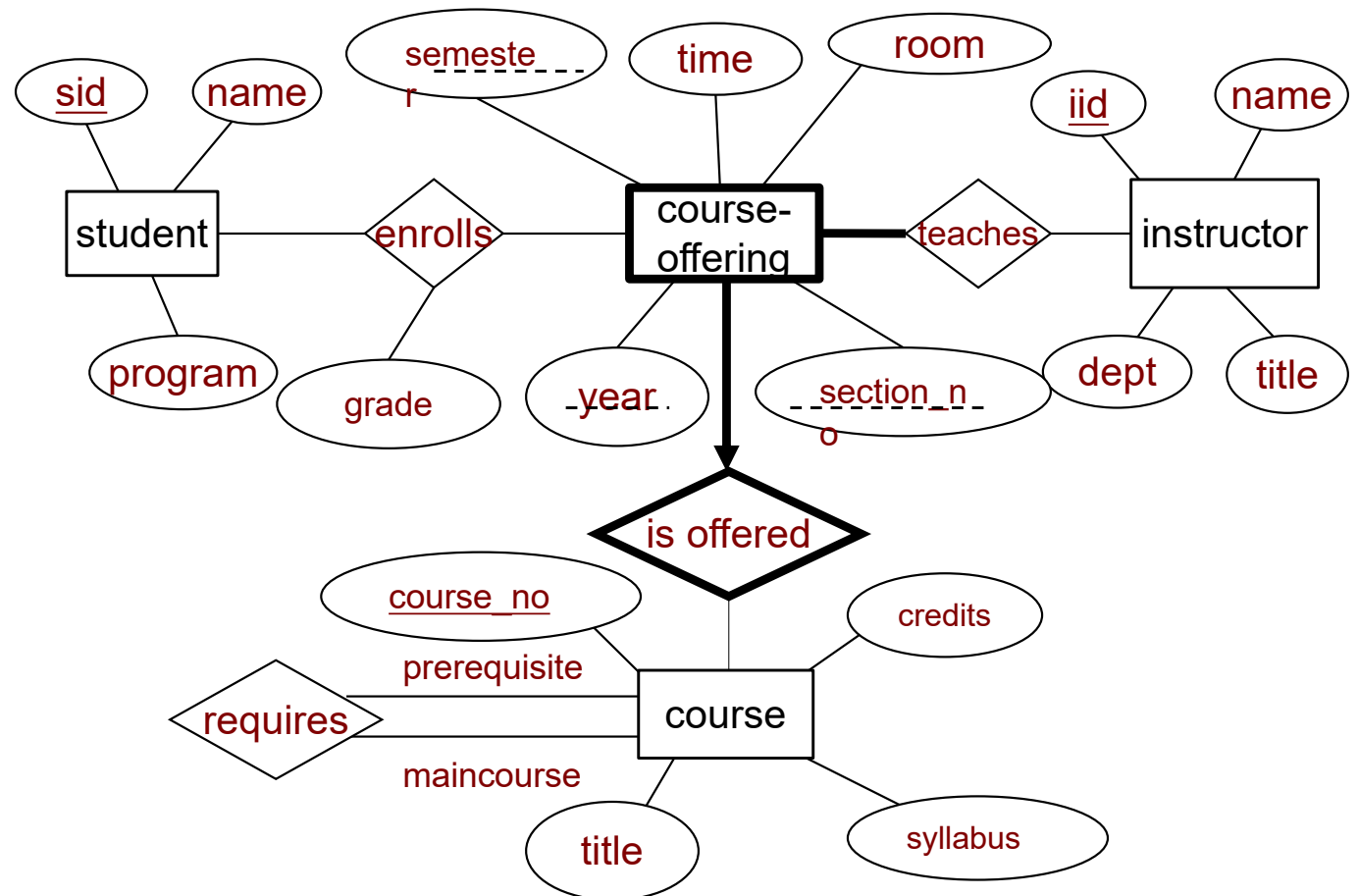
Weak Entity



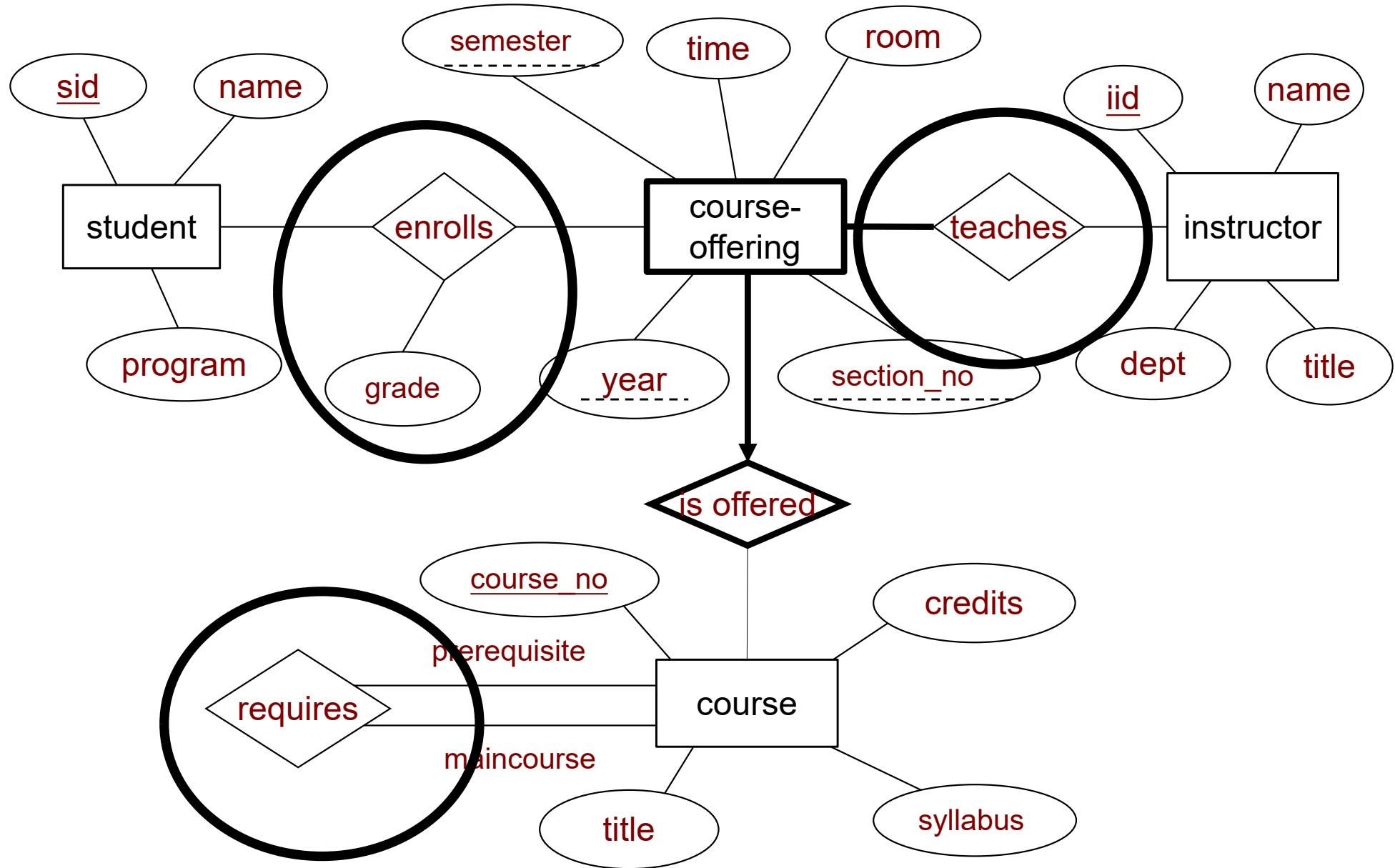
E-R diagram for a university.

Table of Week Entity

course-offerings (course_no, section_no, year, semester,
time, room)



Relationships (without Weak Entities)



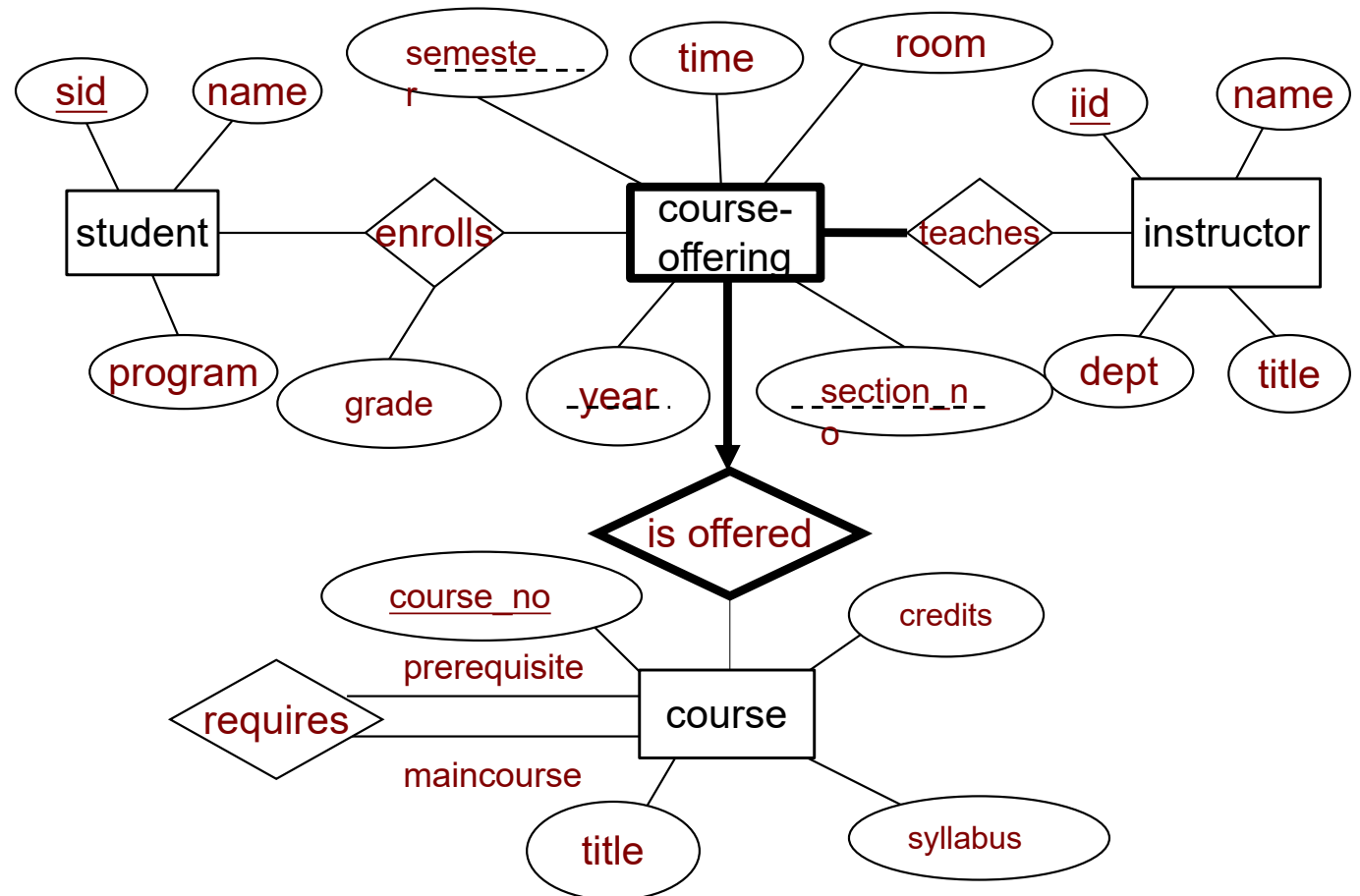
E-R diagram for a university.

Tables of Relationships (without Weak Entities)

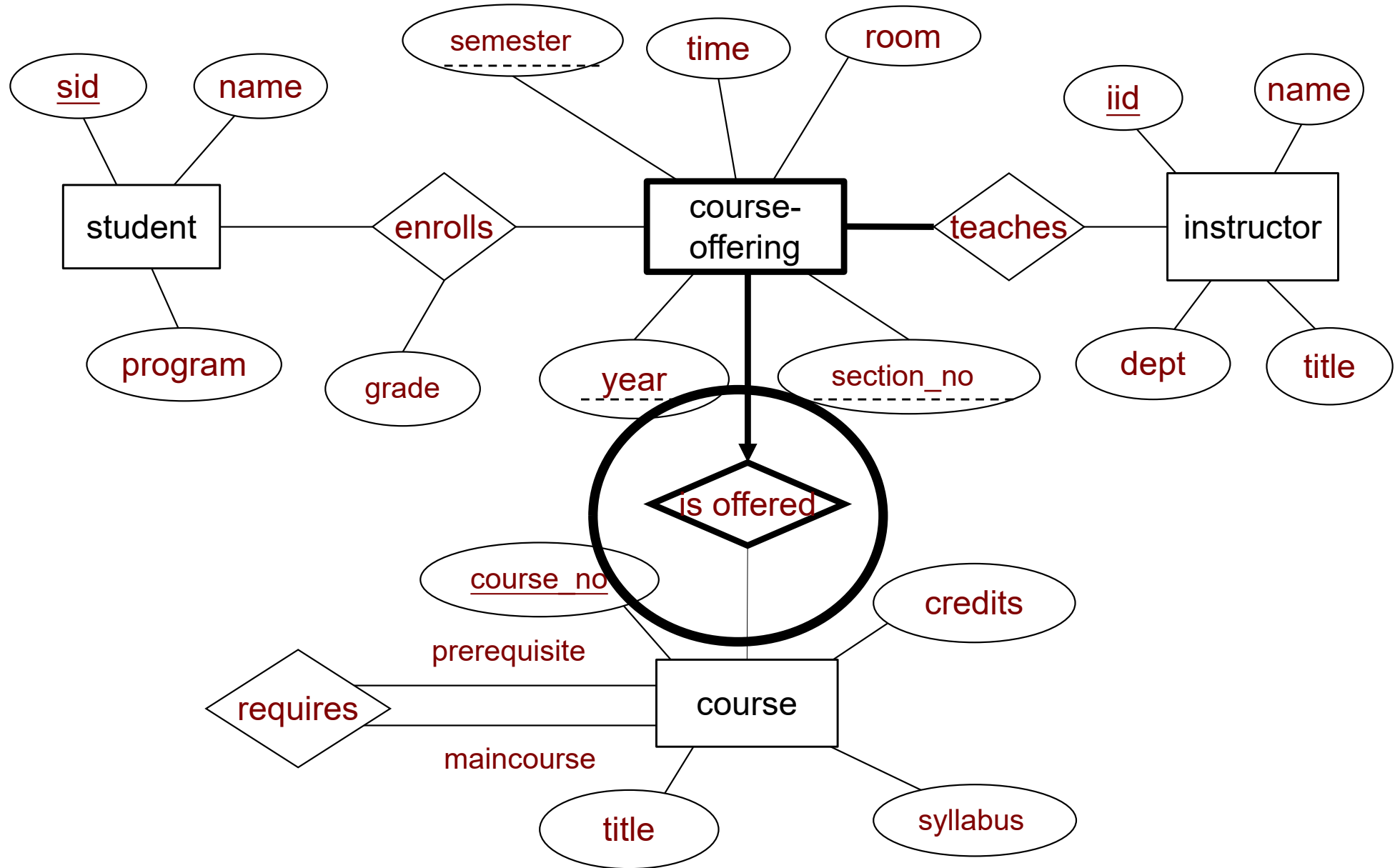
enrols (sid, course_no, section_no, semester, year, grade)

teaches (iid, course_no, section_no, semester, year)

requires (main_course_no, prerequisite_no)



Relationship with Weak Entity



E-R diagram for a university.

Table of Relationship with Weak Entity

- There is no extra table for the relationship between a weak entity and its “owner/strong” entity.
- The relationship is already present in the table for the weak entity.

course-offerings (courseno, secno, year, semester, time, room)

- If the relationship between A and B is many-to-one
 - Include the key of B into the table of A
- If the relationship between A and B is one-to-one
 - Include either A or B’s key into the other’s table